Lab 1: Getting Started with R

Rafael Campos-Gottardo*

January 30 & 31, 2025

Welcome to POLI311

POLI311 builds on the introduction to R provided in POLI210. The focus of this course in on running and interpreting linear regression models in R. R is a free open source programming language designed for statistical computing. Despite having a slightly higher learning curve than other statistical software programs used in the social sciences (e.g. STATA, SPSS, etc.) R has a number of advantages. First, R is free which makes it more accessible. Second, the R is an open source programming language which means that the R community has created a number of packages designed to perform specific statistical analyses. These packages range from packages designed for graphing (`ggplot2`) to more specialized packages designed for specific tasks (e.g. calculating adjusted marginal means for conjoint experiments). Many quantitative political science papers published in the *American Political Science Review* now include R packages (See Mehlhaff, 2023 for an example). Finally, R allows political scientists to share their statistical analyses which increases the reproducibility of research in political science. However, as a programming language R comes with a greater learning curve than other statistics software.

Therefore, in these labs we will cover how to use R for the regression analyses commonly performed in political science. Since we only have 50 minutes together each week it is important to come to lab prepared and attend office hours if you need additional clarification.

Learning Objectives

- Open R studio and create an R project.
- Perform basic mathematical operations in R.
- Create your first R objects.
- Download a data set from *mycourses* and load the data into R as a `data.frame`.
- Understand the structure of a `data.frame`.
- Use the `table()` function to inspect the variables in a `data.frame`.

Before Lab

Before your lab this week please ensure that you have completed the following:

- Read this lab guide.
- Download R and RStudio.
- Familiarize yourself with R Projects. (If you need a refresher on how to create an R project please refer to the following video.)
- Familiarize yourself with your computer's file structure. When working with R it is important to know where files you download go and where you want to store your work for this class.

*This lab guide builds on those created by Aaron Erlich and Colin Scott. I would also like to acknowledge Brendan Szendro, Nicholas Vaxelaire, Guila Cohen, and Natalie Delmonte for their help developing this lab guide.

Performing Basic Mathematical Operations in R

Like most programming languages R can be used as a calculator. In R Studio the console can be used to perform basic mathematical operations. The console is located in the bottom left pane in R studio and has a > and cursor. To perform basic mathematical simply type what mathematical operation you want into the console and press **Enter**.

Basic mathematical operations include addition:

```
3 + 5
```

```
## [1] 8
```

R begins each output with a number in square brackets.

Practice Problem 1: What does the number `[1]` in this output indicate?

You can also perform more complex mathematical operations like multiplication and division:

```
4 * 3
```

```
## [1] 12
```

```
5 / 6
```

```
## [1] 0.8333333
```

R also ignores spaces when executing a command. However, it is important to write code that is readable to others (your professors, TAs, and journal reviewers need to be able to read your code). Therefore, it is best practice to separate numbers and operators with a space.

When performing mathematical operations R respects the order of operations. As a result the following operations will yield different results:

```
(3 + 4) * 5^3 / 7
```

```
## [1] 125
```

```
3 + 4 * 5^3 / 7
```

```
## [1] 74.42857
```

Functions in R

In addition to the following basic mathematical operators (+, -, *, /, ^, %%[1], etc.) R comes with a number of built-in functions to perform mathematical or statistical operations and manipulate data. Functions are denoted by a word (or words) followed by brackets (e.g., `function()`). For example, the `sqrt()` function provides the square root.

```
sqrt(16)
```

```
## [1] 4
```

```
sqrt(80)
```

```
## [1] 8.944272
```

Practice Problem 2: How would you calculate the following formula in R: $\sqrt{\frac{14 \times 54^2}{(55-14)+98 \times 2}}$?

[1] This operator gives you the remainder from division.

Objects in R

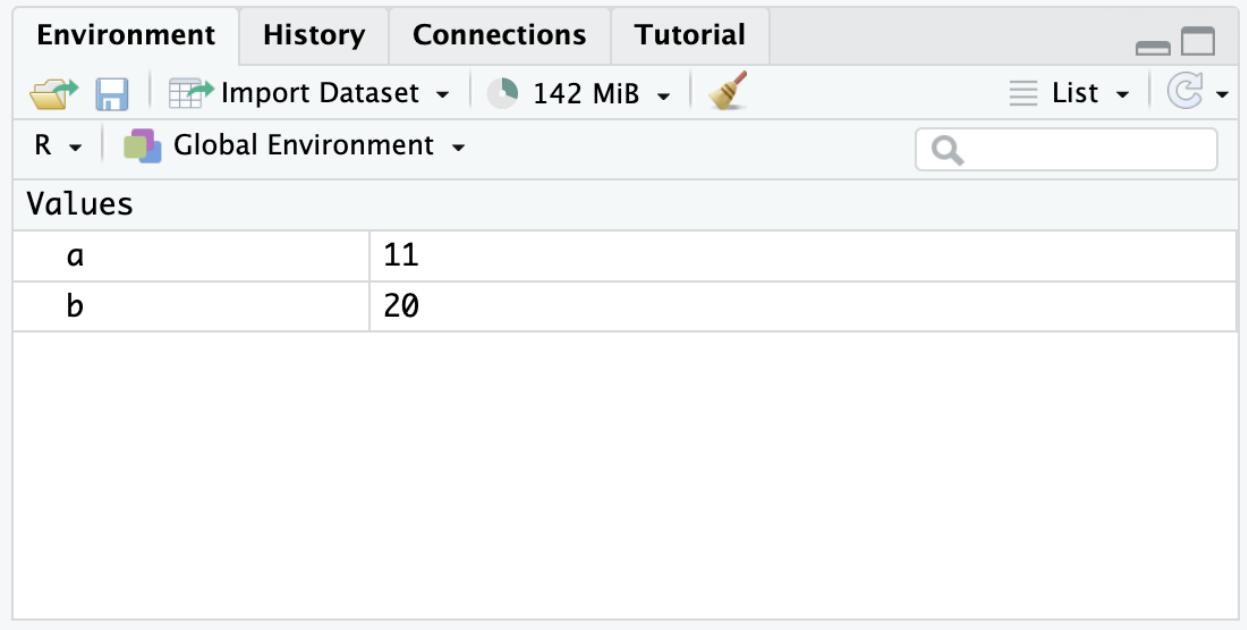
R is an object oriented programming language (OOP) which means that information is stored as objects. In order to store information as an object R uses the <- assignment operator (for those familiar with `Python` = also works as an assignment operator in R). One feature of R is that you left (<-) and right assign (->).

The results from mathematical operations can be assigned to objects:

```
a <- 5 + 6

4 * 5 -> b
```

Notice that when you create an object it appears in the environment in the top right panel in R Studio.



The environment viewer allows you to see all objects and dataframes that are currently in your R environment. If you try to call an object that is not in the R environment you will receive an error.

For example if we tried to call the object `d` we would receive the following error: `Error: object 'd' not found`[2]

If you want more practice with objects refer to the *introduction to objects* lab handout.

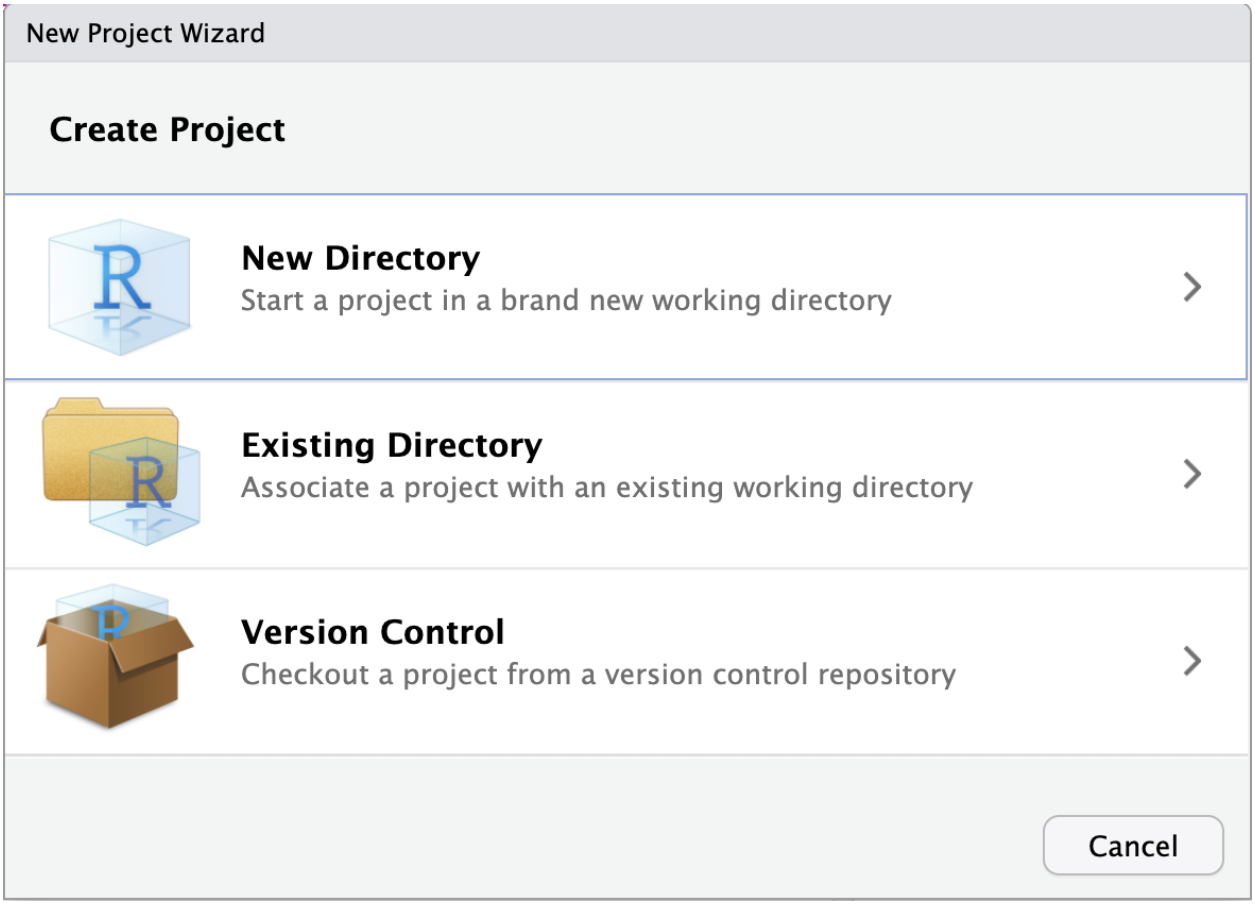
R Projects

Now that we have learned how to create our first objects in R we want to create an R project. An R project helps us organize our files and data. An RStudio project links each R environment to a working directory. Using R projects make it easier to organize and move your work in R and makes your code more reproducible.[3]

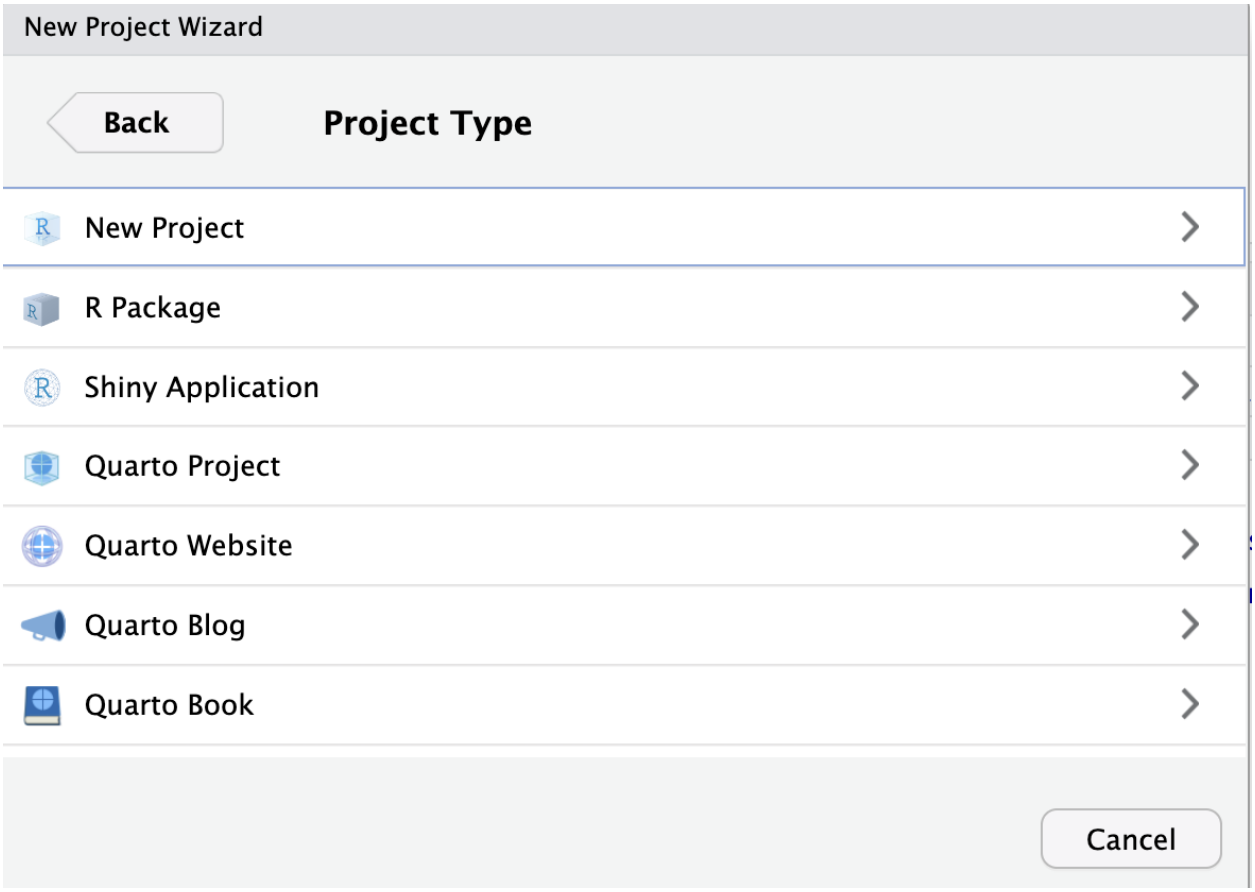
TO create an R Project open R Studio and click `File > New Project...` Once you click `New Project...` the following window will appear:

[2]One of the most important skills when working in R is understanding and debugging error messages. We will discuss error messages during labs and you can refer to the *Troubleshooting R* section in this lab handout for more tips.

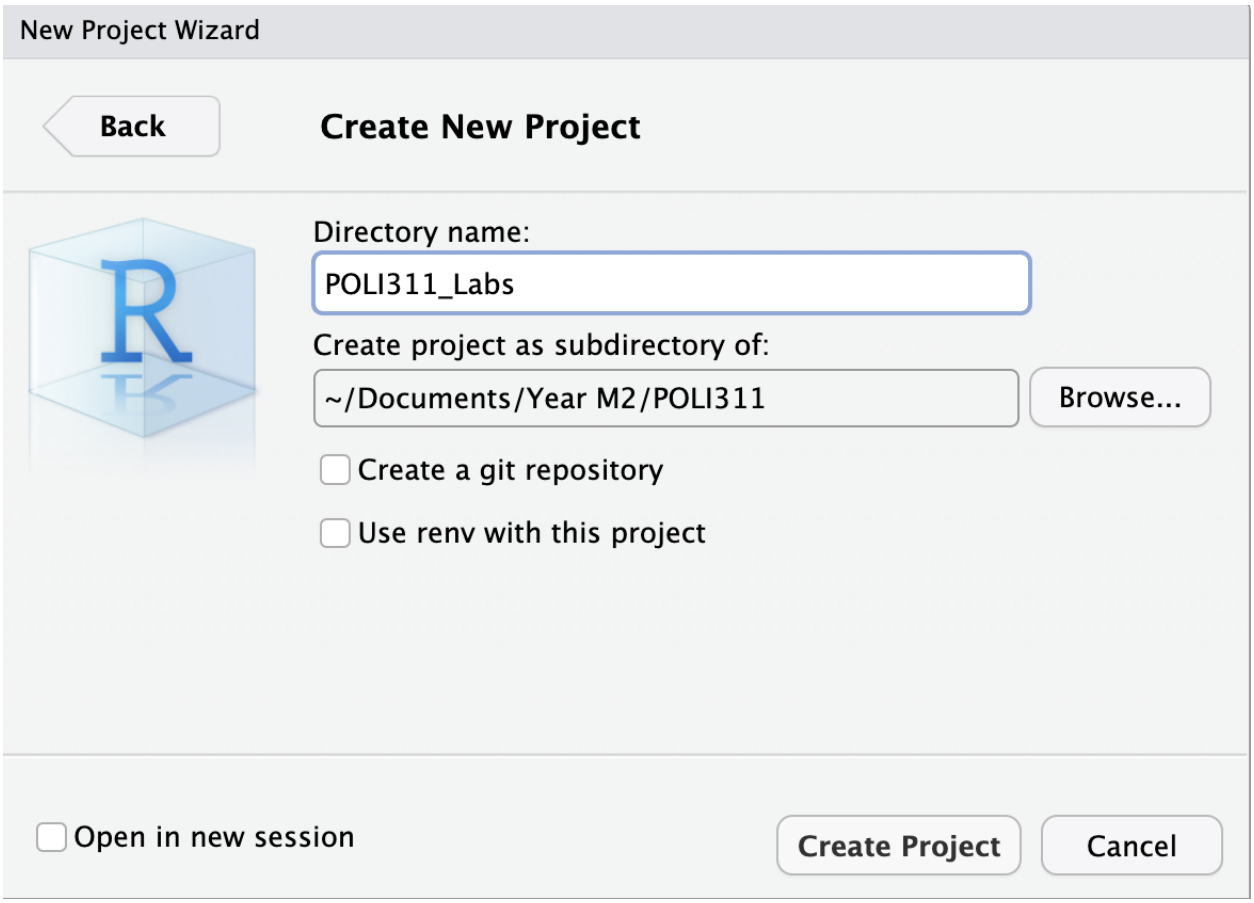
[3]Some researchers prefer using the `setwd()` function. However, this method makes it harder to work in R and makes your code less reproducible. If you are interested in using `setwd()` this video explains how to use `setwd()` and why you should use R Projects instead of `setwd()`.



For now we can select `New Directory` which will create a new folder for our lab files (If you select `Existing Directory` R Studio will create a new `R.Proj` file within an existing folder). Once you select `New Directory` the following window will appear:



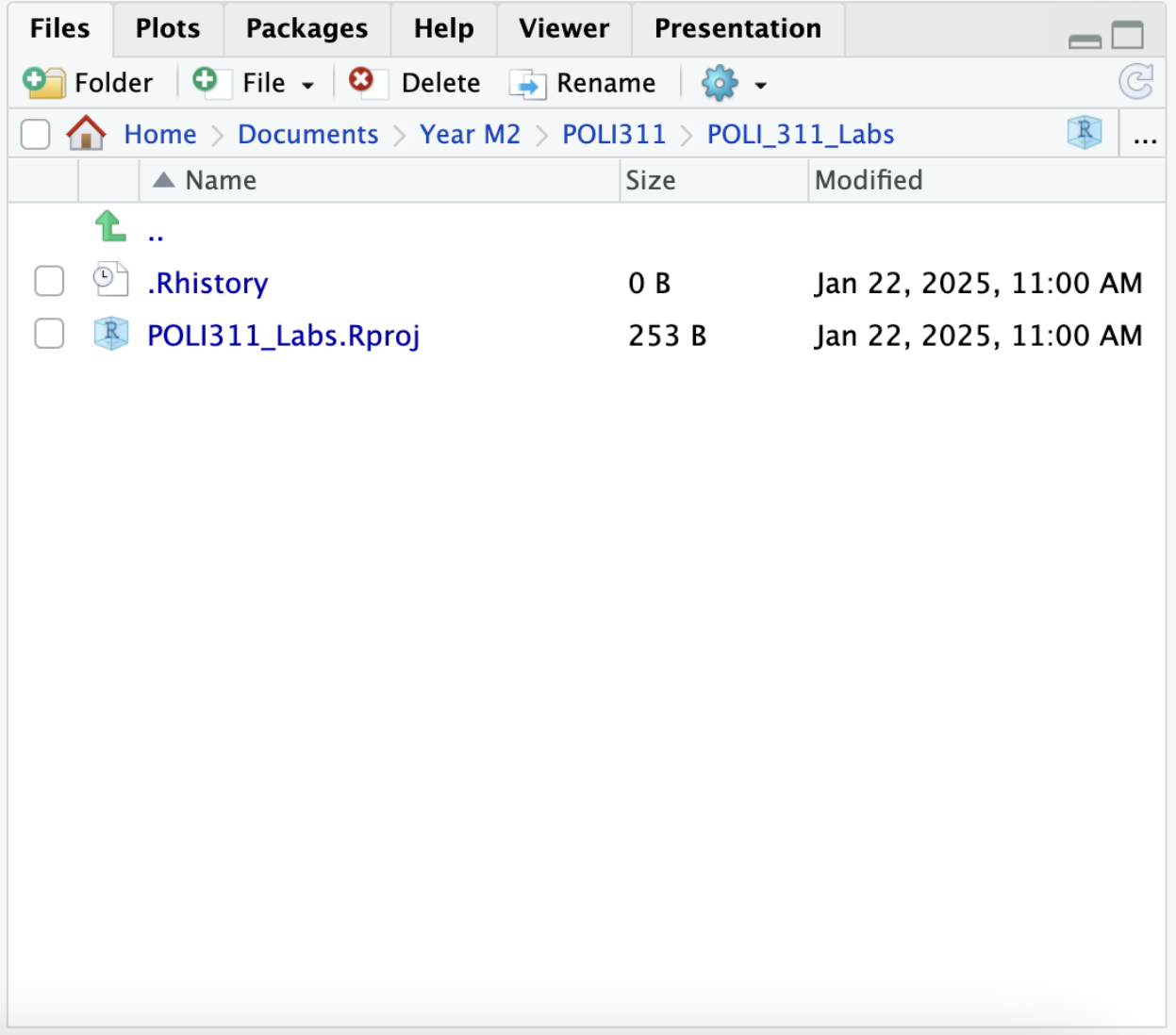
In this window select `New Project`. Finally, you will be given the option to name your new project and select where you want your new directory to be saved on your computer (this is why its important to have a file management system on your computer).



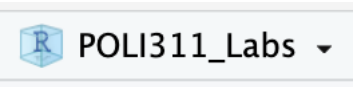
In the `Directory Name` box you can give your project a name, for now let's call our project `POLI311_Labs`. Now R Studio will create a new project that you can use for your labs in this class (I would suggest creating a separate R Project for your final assignment for this class).

Practice Problem 3: Create a new R Project in your POLI311 class folder called `POLI311_Final_Assignment`.

Once you have created a new R Project you will notice a new file in the `Files` panel in the bottom right of R Studio. This is your `.RProj` file which is how you will open your R Project in the future.



Additionally, in the top right corner of R Studio is name of your R Project (**NOTE** if the name of the R Project in the top right corner does not match the `.RProj` file you see in the `Files` panel you are not in the right R Project and the files you see in the file viewer cannot be opened in R).



Importing Data

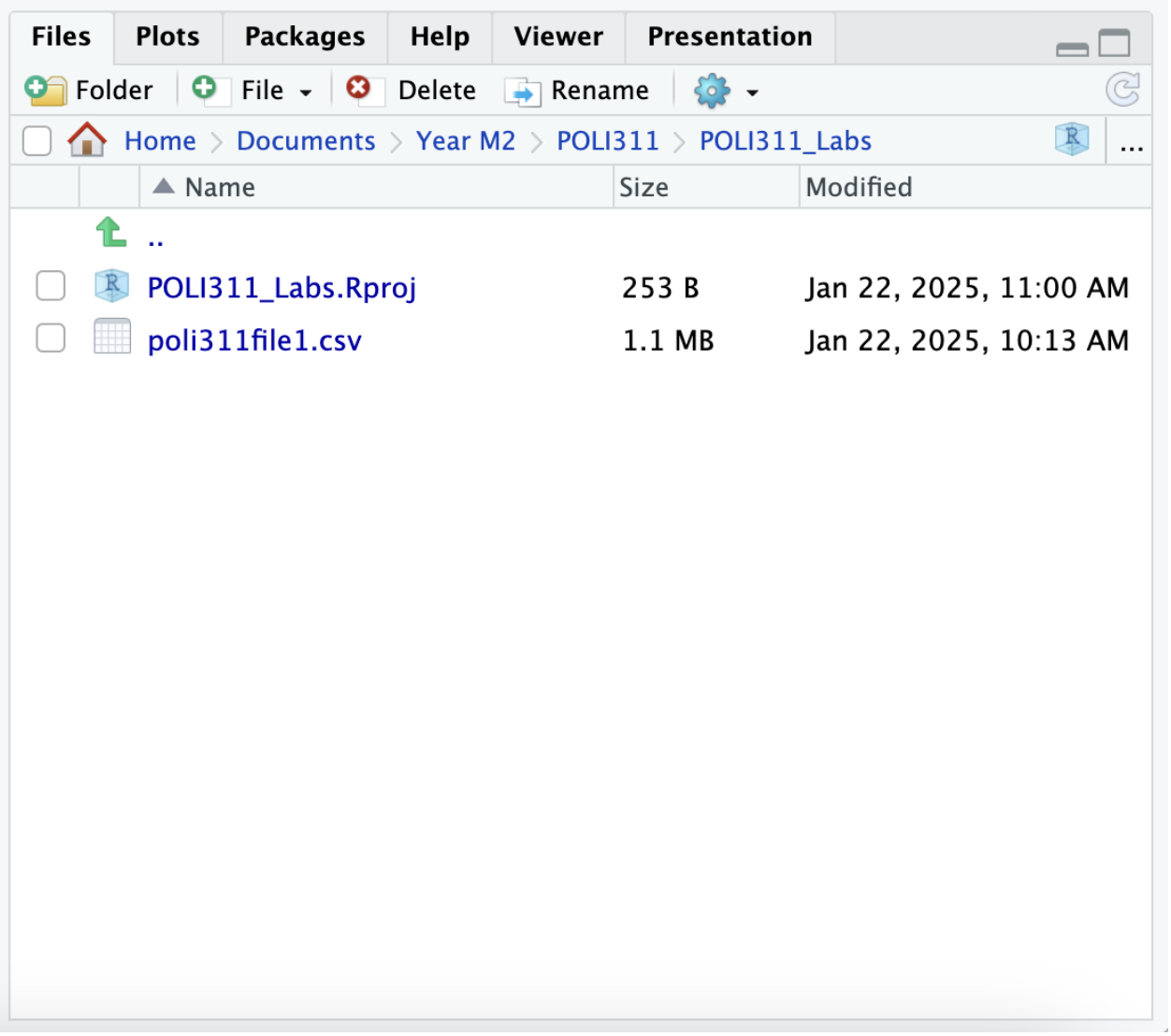
Now that we have created an R Project we can upload data into R. For this lab will be using the `poli311file1.csv` from *mycourses*. Download this file and then locate it in your downloads folder (which is usually found through `Finder/Macintosh HD` on Mac and `File Explorer` on Windows). Next copy `poli311file1.csv` and paste it into your `Poli311_Labs` folder. If you need help locating your `Poli311_Labs` folder you can use the `getwd()` function in R Studio.

```
getwd()
```

```
## [1] "/Users/rafaelc-g/Documents/Year M2/POLI311/POLI311_lab_guides"
```

Which will give you the file path of your R Project folder (**NOTE:** If you are R Studio Cloud the following

video explains how to upload files to the cloud). Once you have have placed the data file into your project's directory you can return to R Studio. You will now notice that there are two files in the `Files` panel.



R Scripts

Now that we have added our data to our directory let's create an R script to save our code. To create an R script select `File > New File > R Script`. Now that you have created an R script you can write your code in the R script like we wrote it in the console. However, you will notice that when you press `Enter` R Studio creates a new line and does not run the current line. In order to, run the current line press `CMD/CNTRL + Enter` with your cursor on the line you want to run. If you want to run multiple lines at once you can highlight all the lines you want to run and then press `CMD/CNTRL + Enter`.

In an R script if you start a line with `#` you can write comments. R will not run these lines as code. Commenting your code is useful for when you need to return to your code later and might not remember what each line of code was for and for submitting assignments in POLI311.

```
# Creating a vector of student names
names <- c("William", "Paris", "Ali", "Natasha", "Maria")
```

In order to save your R script you can press the floppy disk icon below the name of the R script (`Untitled1`).

Let's call our R script for today `Lab1`. You will now notice that in the `Files` panel there is now a file called `Lab1.R`.

Dataframes

One of the most important object types in R is the `dataframe` (or `tibble` when using the `tidyverse`). Dataframes allow you to store two dimensional data (think of a spread sheet) of multiple classes. Dataframes have columns and rows where each row represents a unique observation and each column represents a variable. Unlike a matrix where each column needs to contain the same object class, dataframes can contain multiple classes. To import a `.csv` file into our R environment we can use the `read.csv()` function.

```
df <- read.csv("poli311file1.csv")
```

The `read.csv()` function requires you to write the file name (and path) in quotation marks. (**NOTE** spelling and spacing matter, your file name in quotation marks needs to match exactly with the file name in the folder. To make this process easier R Studio lets you process the files in your working directory by pressing the `tab` key with your cursor in the quotation marks. You cna then use your mouse or the arrow keys and the `return/enter` key to select the data file you want to import.) Make sure you also save your data frame as an object in R. In this case we called the object `df` for data frame.

Now that we have a data file imported we can inspect the data frame with the `str()` function.

```
str(df)
```

```
## 'data.frame':    10078 obs. of  14 variables:
##  $ year              : int  1960 1961 1962 1963 1964 1965 1966 1967 1968 1969 ...
##  $ logpop            : num  0.902 0.903 0.904 0.904 0.905 ...
##  $ loggdp            : num  NA NA NA NA NA NA NA NA NA NA ...
##  $ cowcode           : int  2 2 2 2 2 2 2 2 2 2 ...
##  $ region            : int  NA NA NA NA NA NA NA NA NA NA ...
##  $ v2jupoatck        : num  0.809 0.809 0.809 0.809 0.809 ...
##  $ system            : chr  "" "" "" "" ...
##  $ frac              : num  NA NA NA NA NA NA NA NA NA NA ...
##  $ polity            : num  0.95 0.95 0.95 0.95 0.95 ...
##  $ durable           : num  0.559 0.565 0.571 0.576 0.582 ...
##  $ constitution_age  : num  0.734 0.738 0.742 0.747 0.751 ...
##  $ constitution_scope: num  0.605 0.605 0.605 0.605 0.605 ...
##  $ amendment_rate    : num  0.0674 0.0674 0.0674 0.0674 0.0674 ...
##  $ stabs_rate        : num  0.182 0.182 0.182 0.182 0.182 ...
```

This function displays the class of our dataframe (`data.frame`), the number of observations in our dataset (`10078 obs.`), the number of variables in our data frame (`14 variables`), and a preview of each variable. The variable previews contain the name of the variable (e.g., `year`, `logpop`, `loggdp`, etc.), variable class (e.g., `int`, `num`,`chr`, etc.) and the first 10 values of each variable.

Now that we know more about our dataframe we can begin using our data frame by indexing it. There are two ways to index a dataframe. We can use square brackets (`[]`) or the `$` selection operator. When using `[]` brackets the first argument in the brackets is the row(s) and the second argument is the column(s).

For example the following code selects the 3rd observation (row 3) in the `polity` (column 9) variable.

```
df[3, 9]
```

```
## [1] 0.95
```

We can also select multiple rows/columns using the `:` operator or the `c()` concatenate function. The concatenate function is one of the most used functions in R and allows you to supply multiple values to a single argument in a given function.

To select multiple consecutive rows/columns we can use the `:` operator:

```
df[4:16, 1]
```

```
##  [1] 1963 1964 1965 1966 1967 1968 1969 1970 1971 1972 1973 1974 1975
```

and to select non-consecutive rows/columns we can use the `c()` function.

```
df[5, c(1, 4, 5, 10)]
```

```
##   year cowcode region   durable
## 5 1964       2     NA 0.5823529
```

We can also conbine both methods to select a mix of consecutive and non-consecutive rows/columns.

```
df[3:12, c(1:3, 6)]
```

```
##    year    logpop loggdp v2jupoatck
## 3  1962 0.9037685     NA  0.8091911
## 4  1963 0.9044514     NA  0.8091911
## 5  1964 0.9051106     NA  0.8091911
## 6  1965 0.9057038     NA  0.8091911
## 7  1966 0.9062520     NA  0.8091911
## 8  1967 0.9067687     NA  0.8091911
## 9  1968 0.9072425     NA  0.8091911
## 10 1969 0.9077064     NA  0.6952432
## 11 1970 0.9082592     NA  0.8583714
## 12 1971 0.9088592     NA  0.8583714
```

Using square brackets we can also index datasets by variable name. In order, to index by name we can supply a character vector in the square brackets using single `''` or double `""` quotation marks. You can supply either a single variable name or multiple variable names using the `c()` function.

```
df[1:5, "year"]
```

```
## [1] 1960 1961 1962 1963 1964
```

```
df[1:5, c("year", "cowcode")]
```

```
##   year cowcode
## 1 1960       2
## 2 1961       2
## 3 1962       2
## 4 1963       2
## 5 1964       2
```

Finally, if you want to display all the rows/columns for a given set of rows/columns you can leave the other argument blank in the square brackets.

```
df[1:5, ]
```

```
##   year    logpop loggdp cowcode region v2jupoatck system frac polity   durable
## 1 1960 0.9022518     NA       2     NA  0.8091911     NA    0   0.95 0.5588235
## 2 1961 0.9030386     NA       2     NA  0.8091911     NA    0   0.95 0.5647059
## 3 1962 0.9037685     NA       2     NA  0.8091911     NA    0   0.95 0.5705882
## 4 1963 0.9044514     NA       2     NA  0.8091911     NA    0   0.95 0.5764706
## 5 1964 0.9051106     NA       2     NA  0.8091911     NA    0   0.95 0.5823529
```

```
##   constitution_age constitution_scope amendment_rate stabs_rate
## 1        0.7339056          0.6049383     0.06737801   0.181598
## 2        0.7381975          0.6049383     0.06737801   0.181598
## 3        0.7424893          0.6049383     0.06737801   0.181598
## 4        0.7467811          0.6049383     0.06737801   0.181598
## 5        0.7510729          0.6049383     0.06737801   0.181598
```

```
# df[ , "year"] NOTE this line is commented out because it will output 10,078 obersevations.
```

The other method of indexing a dataframe involves using the `$` operator and the name of a specific variable. This method outputs all the values of a given variable as a vector and is generally used when supplying variables to functions. For example to select the `year` variable from our `df` we would run the following line of code: `df$year`.

Another function we can use for inspecting our data is the `table()` function. This function allows us to create a table that displays the number of observations in each value of a given variable.

```
table(df$year)
```

```
##
## 1960 1961 1962 1963 1964 1965 1966 1967 1968 1969 1970 1971 1972 1973 1974 1975
##  145  146  146  146  146  146  146  146  146  146  146  148  148  148  148  148
## 1976 1977 1978 1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991
##  148  148  148  148  148  148  148  148  148  148  148  148  148  149  164  167
## 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004 2005 2006 2007
##  168  170  170  170  170  170  171  172  172  172  172  172  172  172  172  172
## 2008 2009 2010 2011 2012 2013 2014 2015 2016 2017 2018 2019 2020 2021 2022
##  172  172  172  173  173  173  173  173  173  173  173  173  173  173  173
```

In this table the number of top indicates the year (e.g., 1960) and the number below indicates the number of observations in a given year (e.g., 145).

Practice Problem 4: How many countries were surveyed in 2011?

We can also provide two arguments to the `table()` function which creates a cross table that displays the number of observations taking on both values in the dataset.

```
table(df$region, df$system)
```

```
##
##          Assembly-Elected President Parliamentary Presidential
##   0    0                          33           692           66
##   1  112                          68           281          469
##   2   37                         168           389          297
##   3    2                          60            74          491
##   4   22                         113           129         1266
##   5    0                          68           131          626
```

In this table there are 491 observations that are presidential systems in region 3, with the columns representing the political systems and the rows representing the region.

Practice Problem 5: How many observations are parliamentary systems in region 2?

Tips and Tricks for R

Think of learning R like learning a new language. Many of the rules will seem arbitrary at first but once you understand the underlying logic it becomes easier to to trouble shoot your problems in R. In the mean time I below I have provided some useful tips for troubleshooting issues in R.

1. Use the built in help function in R! If you proceed the name of a function with a question mark it will open the built in help file in the R Studio Help panel (e.g. `?table()`).
2. Google error codes. Whenever, there is a problem with your code R will give you an error in the console. However, when you are first starting out this codes are often confusing. Therefore, you can usually Google errors and find someone else who has already had the same issue.
3. Use Generative AI. ChatGPT is really good at explaining error codes and how to fix them. You can usually paste the error code into ChatGPT and receive an explanation and steps to solve the error. I find prompting ChatGPT in the following way is the most useful: I am coding in R using the **BLANK** package and received the following error: **ERROR**.[4] ChatGPT will generally provide a number of tips to help you fix the error.
4. Come to office hours and ask your TAs for help. We are more than happy to help you understand any content from the labs during office hours.

Practice Problem solutions

1. The number [1] indicates that the first element in that row is the first element in the vector. If the vector outputted onto multiple lines then each line would start with a different number.
2.

```
sqrt((14 * 54^2)/((55 - 14) + 98 * 2))
```

```
## [1] 13.12453
```

3. Ask your TA to double check your answer if you are unsure.
4. 173
5. 389

[4] **NOTE:** ChatGPT is a useful tool for diagnosing errors in R but requires some base knowledge of R to be used properly. Therefore, learning R in a classroom environment is useful even with the availability of Generative AI programs like ChatGPT. Additionally, ChatGPT can be used to help you diagnose error but using GenAI to write the code for your assignments is **considered plagiarism** and will be dealt with according to the plagiarism policy outlined in the syllabus.

Lab 2 - Working with Data Frames

Rafael Campos-Gottardo*

February 6 & 7, 2025

Learning Objectives

1. Load dataframes that are not in the csv format.
2. Summarize data using base R functions.
3. Load the `tidyverse` and use `pipes`.
4. Clean data using the `dplyr` and `stringr` packages.
5. Summarize data using `dplyr`.

Before Lab

1. Read this lab guide.
2. Review downloading packages in R.
3. Read section "3.4 the pipe" of the R for data science textbook. We will be using the pipe (`%>%`) from the `magrittr` package in the `tidyverse`. However, this pipe uses the same logic as the base R pipe (`|>`) discussed in the textbook and can be used interchangeably.
4. Review the `stringr` package documentation and regular expressions.

Working with "Raw Data"

Now that we have learned the basics of R we can start cleaning dataframes and generating descriptive statistics. Let us start by importing the class dataset we created last week. We can download "class_survey_poli311.xlsx" from *mycourses*.

This dataset has 6 variables that are based on the following questions:

Variable	Question
fav_col	What is your favourite colour?
location	Where are you from?
age	How old are you?
interest	How interested are you in politics?
Gender	Whats your gender?
social_use	Approximately how many hours a week do you spend on social media? (please only input numbers)

You will notice that Google forms exports Excel spreadsheets instead of .csv (comma-separated values) files. Most data you will work with in R is not in the .csv format and requires an additional package to import our data into R.

*This lab guide builds on those created by Aaron Erlich and Colin Scott. I would also like to acknowledge Brendan Szendro, Nicholas Vaxelaire, Guila Cohen, and Natalie Delmonte for their help developing this lab guide.

Installing packages

In order to, import Excel files into R we need to install the `readxl` package. To install a package we use the `install.packages()` function with the name of the package in quotation marks `install.packages("readxl")`. This function allows us to download packages from CRAN (The Comprehensive R Archive Network) into the libraries folder on your computer. CRAN includes all officially approved R packages. However, some user created packages have not been officially approved by the R team and these packages can be installed from github. Github packages usually have more "niche" uses and contain less documentation.

To install the `readxl` package you can run the following lines of code.

```
# The install.packages function installs the package on your computer
# install.packages("readxl")

# Use the library function to load packages
library(readxl)

# If you are curious where R install packages you can run the libParths function
.libPaths()
```

```
## [1] "/Users/rafaelc-g/Library/R/arm64/4.4/library"
## [2] "/Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/library"
```

When installing packages you only have to run the `install.packages()` function once. It will typically cause an error if you try to run it more than once. Therefore, after you run it you can delete it or comment it out. **However**, you have to run the `library()` function every time you restart R or change R projects. If you get the error `Error in read_xlsx() : could not find function "read_xlsx"` it means that you have not loaded the package yet for this R session.

There are a number of other packages that can be used to load data into R. The `haven` package lets you load SPSS (.sav) files with the `read_sav()` function and STATA (.dta) files with the `read_dta()` function.

Practice Problem 1: Install and load the `haven` package in R.

Loading Data

Now we can load the class dataset using the `read_xlsx()` function.

```
class_df <- read_xlsx("class_survey_poli311.xlsx")
```

Now we can start working with our new dataset. We can use the `head()` function to look at the first 5 rows of our dataframe and the `str()` function to look at the variables.

```
head(class_df)
```

```
## # A tibble: 6 x 6
##   fav_col location                    age interest Gender social_use
##   <chr>   <chr>                     <dbl>    <dbl> <chr>  <chr>
## 1 purple  The United States            19        4 Woman  21
## 2 Red     The United States            19        5 Man    19
## 3 Pink    Another Canadian Province    21        3 Woman  14
## 4 Purple  Another Canadian Province    21        4 Woman  8
## 5 Purple  Another Canadian Province    19        5 Woman  10
## 6 <NA>    Another Canadian Province    20        5 Woman  10
```

```
str(class_df)
```

```
## tibble [45 x 6] (S3: tbl_df/tbl/data.frame)
##  $ fav_col   : chr [1:45] "purple" "Red" "Pink" "Purple" ...
```

```
##  $ location  : chr [1:45] "The United States" "The United States" "Another Canadian Province" "Anothe
##  $ age       : num [1:45] 19 19 21 21 19 20 20 20 20 20 ...
##  $ interest  : num [1:45] 4 5 3 4 5 5 5 5 5 5 ...
##  $ Gender    : chr [1:45] "Woman" "Man" "Woman" "Woman" ...
##  $ social_use: chr [1:45] "21" "19" "14" "8" ...
```

Descriptive Statistics

We are also interested in generating descriptive statistics. One way we can do so is using the `summary()` function. If we use the `summary` function on the whole dataset it will generate the mean, median and quartiles for all the numeric variables in our dataframe.

```
summary(class_df)
```

```
##    fav_col            location             age           interest
##  Length:45          Length:45          Min.   :18.00   Min.   :2.000
##  Class :character   Class :character   1st Qu.:19.00   1st Qu.:4.000
##  Mode  :character   Mode  :character   Median :20.00   Median :5.000
##                                        Mean   :19.98   Mean   :4.545
##                                        3rd Qu.:20.00   3rd Qu.:5.000
##                                        Max.   :26.00   Max.   :5.000
##                                        NA's   :1       NA's   :1
##     Gender           social_use
##  Length:45          Length:45
##  Class :character   Class :character
##  Mode  :character   Mode  :character
##
##
##
##
```

We can also generate the mean, median, and standard deviation using individual functions.

We can start by looking at the age variable. Remember we can use the `$` operator to extract specific variables from a data frame.

```
mean(class_df$age)
```

```
## [1] NA
```

```
median(class_df$age)
```

```
## [1] NA
```

```
sd(class_df$age)
```

```
## [1] NA
```

Why do we get NA for the mean, median, and standard deviation?

We can use the unique function to explore our variables and see what is causing this error. The `unique()` function displays all the unique values of a variable.

```
unique(class_df$age)
```

```
## [1] 19 21 20 18 22 23 26 NA
```

Practice Problem 2: Please only try this problem if you want more practice with more advanced features in R. Try to run the unique function on each variable using a loop.

As we can see there are NA values in the age variable. Therefore, we need to add an additional argument to the `mean()`, `median()`, and `sd()` functions to indicate that we want the function to ignore the `NA` values. By default the `na.rm` argument equals `FALSE`.[1] We need to specify `na.rm = TRUE`. Now the `mean()`, `median()`, and `sd()` functions will generate the correct values.

```
mean(class_df$age)
```

```
## [1] NA
```

```
median(class_df$age)
```

```
## [1] NA
```

```
sd(class_df$age)
```

```
## [1] NA
```

The Tidyverse

Now that we have generated some basic descriptive statistics we can clean our dataset more using the tidyverse suite of packages. These packages provide very useful tools for data science in R and share a common "grammer." We can load the `tidyverse` the same way as the `readxl` package.

```
#install.packages("tidyverse")
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become error
```

One of the most useful features of the `tidyverse` is the pipe (`%>%`) from the `magrittr` package. The pipe lets you insert the previous line into the first argument of the function on next line of code. The most common usage of the pipe is outlined below. One of the most useful features of the pipe is that it improves the readability of your code and lets you easily use multiple functions at once.

```
# Without the pipe

other_fuction(data, function(arguement2))

# With the pipe

data %>%
  function() %>%
  other_function(arguement2)
```

If you are still unsure about the pipe we will discuss the pipe further as we discuss the `tidyverse`.

[1] If we look at the help file for the mean function (`?mean`), we can see the default options for this function `## Default S3 method: mean(x, trim = 0, na.rm = FALSE, ...)`

Cleaning Data

Now that we have imported our data we can clean it using functions from the dplyr package in the tidyverse. Dplyr uses the grammar of the tidyverse and contains a number of functions that can be used for data manipulation. We will go over the main dplyr functions one by one to clean our data set.

Mutate

The mutate() function adds new variables to a data frame that are a function of existing variables. The new variable will either be added as a new row in your data frame or replace an existing variable if you use the same name. Let us use the mutate function to fix the social media usage variable. You will notice that some respondents included words in their response to the social media usage question.

We want to make this variable numeric so we can find the average number of hours 210 students spend on social media. In order to, extract just the numbers from the responses we use the mutate() function and str_extract() function from the stringr package. This function allows us to extract information from string (character) variables that match a certain pattern. The pattern we want to match is "\\d+" which is the first occurrence of a digit. The \\d is the regex (regular expression) short hand for digits and the + indicates that you want R to match one or more of the preceding character.

```
# Let's pipe in our dataset and assign back into our dataset to save it
class_df <- class_df %>%
# Let's use a new name to not override the old variable
  mutate(social_use_num = str_extract(social_use, "\\d+"),
# Now we need to transform it into a numeric variable
# Let's override the previous version
         social_use_num = as.numeric(social_use_num))
```

Summarize

Now we can use the summarize function in dplyr to create a new data frame of summary information. We want the min, max, mean, median, and standard deviation.

```
class_df %>%
  summarize(Mean = mean(social_use_num, na.rm = TRUE),
            Median = median(social_use_num, na.rm = TRUE),
            `Standard Deviation` = sd(social_use_num, na.rm = TRUE),
            Min = min(social_use_num, na.rm = TRUE),
            Max = max(social_use_num, na.rm = TRUE))
```

```
## # A tibble: 1 x 5
##    Mean Median `Standard Deviation`   Min   Max
##   <dbl>  <dbl>                <dbl> <dbl> <dbl>
## 1  9.91    8.5                 6.16     2    35
```

Now we have produced a nice summary table using the summarize() function. We can also breakdown our descriptive statistics by gender using the group_by() function. We follow the same syntax but add a separate line using the pipe that includes the group_by() function.

```
class_df %>%
  group_by(Gender) %>%
  summarize(Mean = mean(social_use_num, na.rm = TRUE),
            Median = median(social_use_num, na.rm = TRUE),
            `Standard Deviation` = sd(social_use_num, na.rm = TRUE),
            Min = min(social_use_num, na.rm = TRUE),
            Max = max(social_use_num, na.rm = TRUE))
```

```
## # A tibble: 3 x 6
```

```
##   Gender  Mean Median `Standard Deviation`   Min   Max
##   <chr>  <dbl>  <dbl>                <dbl> <dbl> <dbl>
## 1 Man    12.9      10                 9.77     3    35
## 2 Woman   9.34      8                 4.39     3    21
## 3 <NA>    4         4                 2.83     2     6
```

In future weeks we will discuss how to turn these tables into markdown or latex tables that can be used in your assignments and research papers.

Select

The next important `dplyr` function is the `select()` function. This function picks out variables from your data frame using their names. For example if we want to create a new data frame that does not include the "favourite colour" variable we can do so using the `select()` function.

```
class_df_no_col <- class_df %>%
  select(-fav_col) # the - indicates that we want to exclude that variable

head(class_df_no_col)
```

```
## # A tibble: 6 x 6
##   location                    age interest Gender social_use social_use_num
##   <chr>                     <dbl>    <dbl> <chr>  <chr>               <dbl>
## 1 The United States            19        4 Woman  21                     21
## 2 The United States            19        5 Man    19                     19
## 3 Another Canadian Province    21        3 Woman  14                     14
## 4 Another Canadian Province    21        4 Woman  8                       8
## 5 Another Canadian Province    19        5 Woman  10                     10
## 6 Another Canadian Province    20        5 Woman  10                     10
```

```
# This code only creates a data frame that only includes the favourite colour variable
class_df_col <- class_df %>%
  select(fav_col)

head(class_df_col)
```

```
## # A tibble: 6 x 1
##   fav_col
##   <chr>
## 1 purple
## 2 Red
## 3 Pink
## 4 Purple
## 5 Purple
## 6 <NA>
```

Practice Problem 3: What could have caused this error: `Error in select()! Can't select columns that don't exist. Column fav_col doesn't exist.`?

Filter

Finally, we want to use the `filter()` function to subset the data. This function selects rows that match a certain condition. For example, if we want a subset of our data frame that only includes the individuals in our class who are the most interested in politics (5) we can use the following code.

```
most_interested_df <- class_df %>%
  filter(interest == 5)
```

Notice that we use == instead of = to test for equality (whether one value equals another). We use two equal signs because one equals sign is another assignment operator that works the same as the arrow we usually use.

Back to pipes

Now is a good time to show you the usefulness of the pipe. We can perform all the cleaning code we just did in one go with and without the pipe.

```
# With the pipe
class_df %>%
  mutate(social_use_num = str_extract(social_use, "\\d+"),
         social_use_num = as.numeric(social_use_num)) %>%
    select(-fav_col) %>%
  filter(interest == 5) %>%
    group_by(Gender) %>%
  summarize(Mean = mean(social_use_num, na.rm = TRUE),
            Median = median(social_use_num, na.rm = TRUE),
            `Standard Deviation` = sd(social_use_num, na.rm = TRUE),
            Min = min(social_use_num, na.rm = TRUE),
            Max = max(social_use_num, na.rm = TRUE))
```

```
## # A tibble: 3 x 6
##   Gender  Mean Median `Standard Deviation`   Min   Max
##   <chr>  <dbl>  <dbl>                <dbl> <dbl> <dbl>
## 1 Man    13.1      10                10.2      3    35
## 2 Woman   8.16      8                 3.96     3    21
## 3 <NA>    6         6                NA        6     6
```

```
# Without the pipe
  summarize(
    group_by(
    filter(
    select(
      mutate(
        class_df,
        social_use_num = str_extract(social_use, "\\d+"),
         social_use_num = as.numeric(social_use_num)),
      -fav_col),
    interest == 5),
    Gender),
            Mean = mean(social_use_num, na.rm = TRUE),
            Median = median(social_use_num, na.rm = TRUE),
            `Standard Deviation` = sd(social_use_num, na.rm = TRUE),
            Min = min(social_use_num, na.rm = TRUE),
            Max = max(social_use_num, na.rm = TRUE))
```

```
## # A tibble: 3 x 6
##   Gender  Mean Median `Standard Deviation`   Min   Max
##   <chr>  <dbl>  <dbl>                <dbl> <dbl> <dbl>
## 1 Man    13.1      10                10.2      3    35
## 2 Woman   8.16      8                 3.96     3    21
## 3 <NA>    6         6                NA        6     6
```

Notice how the code that does not use the pipe is more confusing than the code that uses the pipe (even with the hard returns). The code without the pipe is also harder to write because it is less intuitive. Without the pipe you have to put the last function first instead of the first function first. The pipe is also useful because

it allows you to separate each function with a line and place the functions in order.

Practice Problem Answers

Practice Problem 1:

```
# install.packages("haven") #Uncomment this line to load the package

library(haven)
```

Practice Problem 2:

```
variables <- names(class_df)

for(i in 1:length(variables)){
  print(unique(class_df[, variables[i]]))
}
```

```
## # A tibble: 20 x 1
##    fav_col
##    <chr>
##  1 purple
##  2 Red
##  3 Pink
##  4 Purple
##  5 <NA>
##  6 Green
##  7 pink
##  8 navy blue
##  9 Blue
## 10 Navy Blue
## 11 Navy blue
## 12 blue
## 13 Yellow
## 14 yellow
## 15 Burgundy
## 16 green
## 17 Camel, black
## 18 Cyan
## 19 Forest Green
## 20 Hot Pink
## # A tibble: 6 x 1
##   location
##   <chr>
## 1 The United States
## 2 Another Canadian Province
## 3 Quebec
## 4 Europe
## 5 Australia
## 6 <NA>
## # A tibble: 8 x 1
##     age
##   <dbl>
## 1    19
## 2    21
## 3    20
```

```
## 4    18
## 5    22
## 6    23
## 7    26
## 8    NA
## # A tibble: 5 x 1
##   interest
##      <dbl>
## 1        4
## 2        5
## 3        3
## 4        2
## 5       NA
## # A tibble: 3 x 1
##   Gender
##   <chr>
## 1 Woman
## 2 Man
## 3 <NA>
## # A tibble: 21 x 1
##    social_use
##    <chr>
##  1 21
##  2 19
##  3 14
##  4 8
##  5 10
##  6 3
##  7 35
##  8 5
##  9 7
## 10 20
## # i 11 more rows
## # A tibble: 17 x 1
##    social_use_num
##             <dbl>
##  1             21
##  2             19
##  3             14
##  4              8
##  5             10
##  6              3
##  7             35
##  8              5
##  9              7
## 10             20
## 11             16
## 12              6
## 13             15
## 14              4
## 15              9
## 16             NA
## 17              2
```

Practice Problem 3:

This error occurs when you remove a variable from a dataset and then try to select that variable. For example:

```
class_df_no_col <- class_df %>%
  select(-fav_col) # the - indicates that we want to exclude that variable

head(class_df_no_col)

# This code only creates a data frame that only includes the favourite colour variable
 class_df_no_col %>%
  select(fav_col)
```

This code will cause that error to occur.

Lab 3: Linear Regression

Rafael Campos-Gottardo*

2025-03-22

Linear Regression

Linear regression represents one of the most important tools in a political scienctists quantitative toolkit. Linear regression is used by most political scienctists for descriptive, preditive, and causal inference. At its most basic linear regression is a tool to summarize how the mean of an outcome variable (Y) changes based on a linear function of predictor variables ($\mathbf{X}$). In other words regression estimates the difference in the mean outcome variable resulting from a one-unit change in the predictor variables. We can use our class dataset to illustrate this feature of regression.

Let's look at the difference in means of our social media usage variable by gender. We can use the summary table from our `lab3.R` script.

```
mean_social_use
```

```
## # A tibble: 3 x 5
##   Gender  Mean Standard_Deviation   Min   Max
##   <chr>  <dbl>              <dbl> <dbl> <dbl>
## 1 Man    12.9                9.77     3    35
## 2 Woman   9.34               4.39     3    21
## 3 <NA>    4                  2.83     2     6
```

Let's extract the mean value of social media usage for the men and women in our class and find the difference in means.

```
mean_social_use$Mean[2] - mean_social_use$Mean[1]
```

```
## [1] -3.55625
```

This tells us that on average the men in this class use social media for 3.56 more hours a week than the women. However, calcuating the difference in means using this method does not tell us whether this difference in means is statistically signficant. In order to determine that this difference is signficantly different from 0, we can use a bivariate linear regression model to calcuate the standard error and p-value. To fit a linear regression model in R we use the `lm()` function.

The `lm()` function by default requires two arguements first it requires a formula object and second it requires a data arguement. A formula object includes a set of one or more X variables and a Y variable separated by a tilda (`~`). The basic struture of a bivariate formula object is `Y ~ X` and a multivariate model is `Y ~ X_1 + X_2 + X_3`. We can write the formula directly into the regression model or save it first as a formula object.

```
# We can save our formula object
bivariate_formula <- social_use ~ Gender

# And use the class function to see that this is a formula
class(bivariate_formula)
```

*This lab guide builds on those created by Aaron Erlich and Colin Scott. I would also like to acknowledge Brendan Szendro, Nicholas Vaxelaire, Guila Cohen, and Natalie Delmonte for their help developing this lab guide.

```
## [1] "formula"
```

Now that we have created a formula object we can use the `lm()` function to regress `social_use` (Y) on `Gender` (X).

```
bivariate_model <- lm(bivariate_formula, data = class_df)

# Now we can use the summary function to summarize our model

summary(bivariate_model)
```

```
##
## Call:
## lm(formula = bivariate_formula, data = class_df)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -9.9000 -2.9000 -1.3437  0.6563 22.1000
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   12.900      1.908   6.761 4.04e-08 ***
## GenderWoman   -3.556      2.186  -1.627    0.112
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.033 on 40 degrees of freedom
##   (3 observations deleted due to missingness)
## Multiple R-squared:  0.06207,    Adjusted R-squared:  0.03862
## F-statistic: 2.647 on 1 and 40 DF,  p-value: 0.1116
```

What do we notice about our regression model?

We will go through this output line by line.

1. The call at the top repeats the code we used to fit our regression model.
2. The residuals indicate the distance between each point and our regression line. When we graph our regression model this will become more clear.
3. The coefficients are the most important part of a regression model for interpretation. The `(Intercept)` tells us the value of our dependent variable when our X variable is equal to 0. The `GenderWoman` coefficient tells us the difference in the mean value from man (the reference category) to woman. The estimate column tells us the actual values. The estimate for the intercept indicates that the average value of social use for men (the reference category) is 12.90. The estimate for the `GenderWoman` coefficient tells us that the average value for women is 3.56 hours lower then for men. The `Std. Error` is used to calculate the `t.value` and confidence intervals. The `t.value` is used to calculate the `p-value`. Finally, the p-value (`Pr(>|t|)`) indicates the probability that the coefficient is statistically different from 0 based on the standard error. You will notice in this model that the Gender coefficient is not different from 0. We will discuss the Estimates in more detail shortly.
4. The `Signif. codes` provide a short hand that can be used to determine if a variable is statistically significant. The general rule of thumb is that if a variable has at least one star next to it then the coefficient is statistically different from 0.
5. The `Residual standard error` is a measure of variation of the residuals. Residuals are the distance

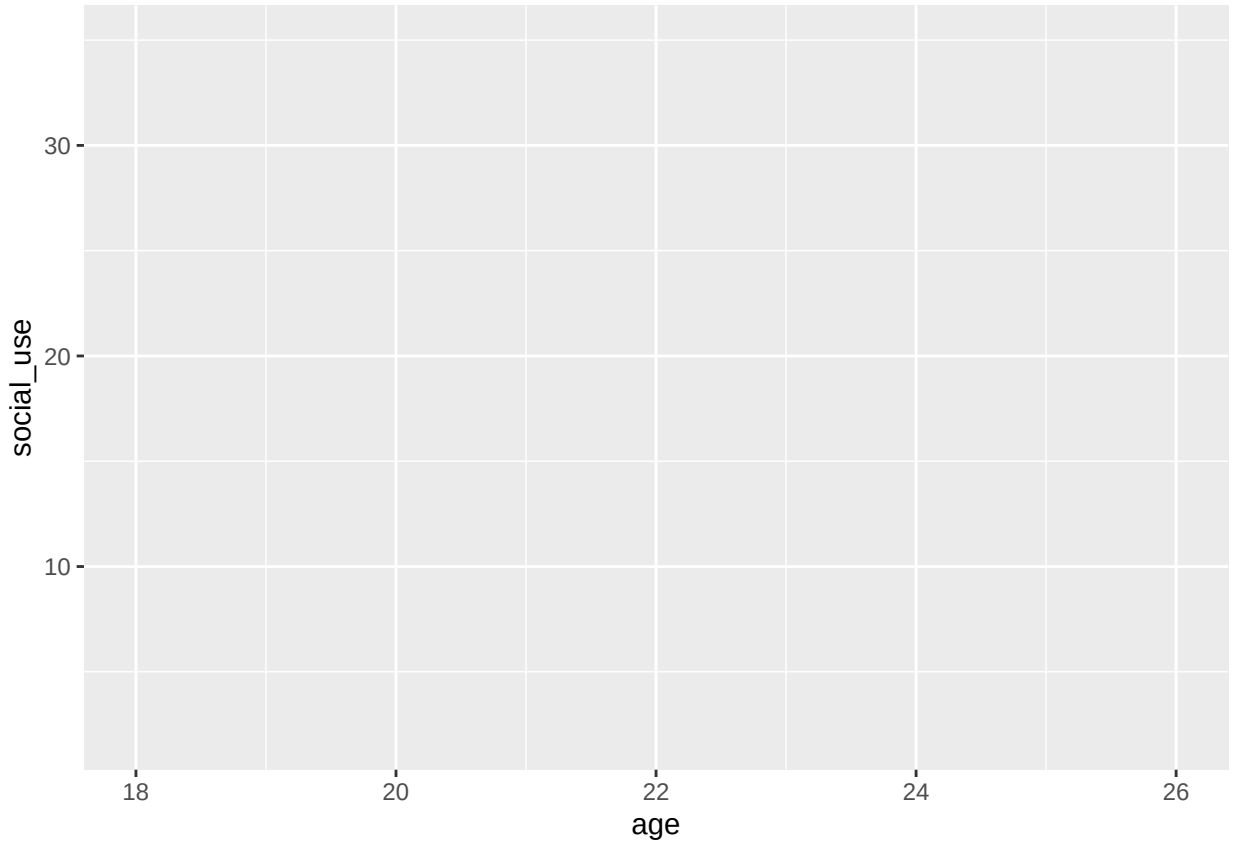
Visualization of Regression Models

In order to better understand the coefficients we can visualize the difference in means using a graph.

Most graphing in R is done using a package called `ggplot`, which is part of the `tidyverse`. However, `ggplot` was developed before the `tidyverse` added the pipe (`%>%`) therefore it uses a plus (`+`) to serve the same purpose. This is an odd feature of `ggplot` that may take some getting used.

To use `ggplot` we always start by piping the dataset into the `ggplot()` function. In the `ggplot()` function we can then define our X and Y variables using the `aes()` function. For this graph our X variable is Gender and our Y variable is social media usage. We are also going to define the colour parameter so that the points in our graph are coloured by gender for easier visualization.

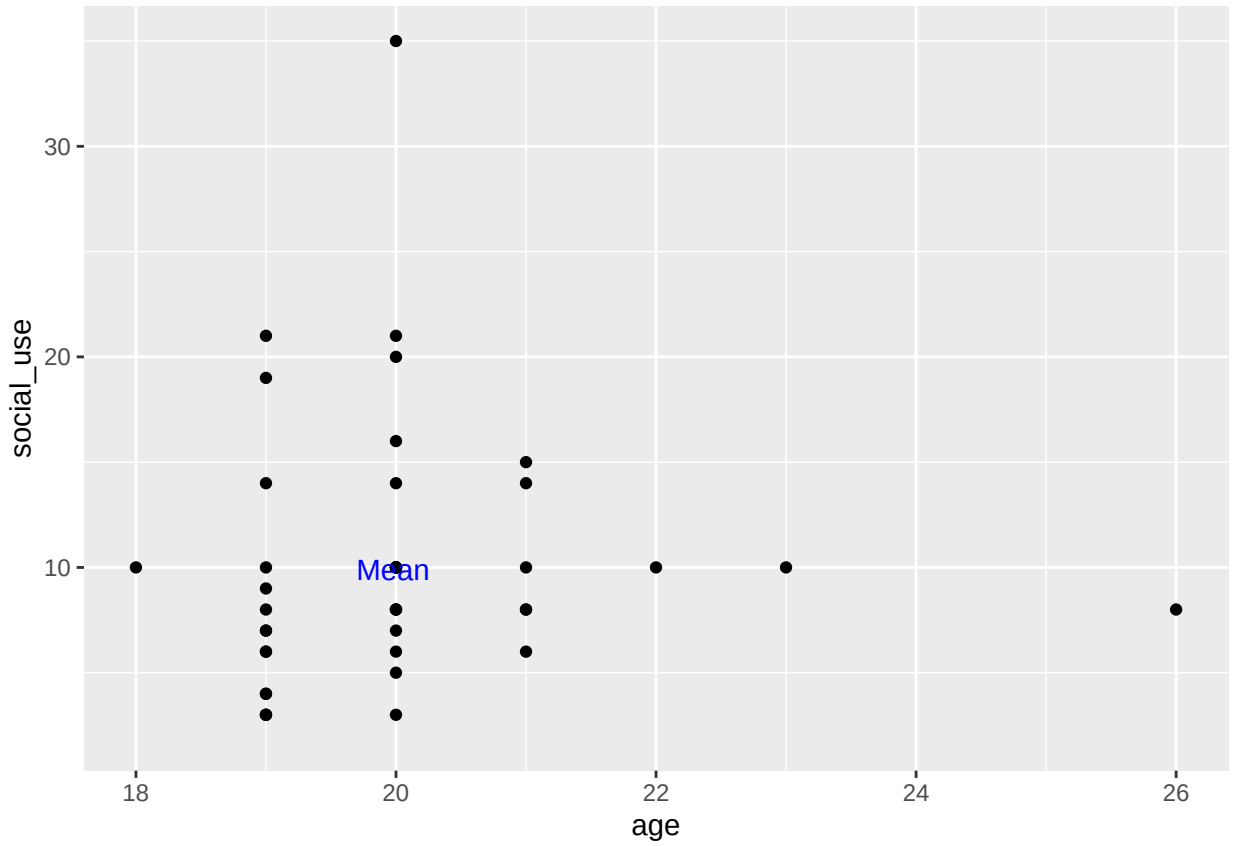
```
class_df %>%
  ggplot(aes(x = age, y = social_use, colour = Gender))
```



You will notice that this code produces a blank graph. This is because we have not specified any geom layers for our graph. We want to make a scatterplot with a regression line on it. To do this we use the `geom_point()` function and add it to the graph using the `+`. We also added `Mean` which is at the mean of X and Y.

```
class_df %>%
  ggplot(aes(x = age, y = social_use)) +
  geom_point() +
  annotate("text", x = mean(class_df$age, na.rm = TRUE),
           y = mean(class_df$social_use, na.rm = TRUE),
           label = "Mean", col = "blue")
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



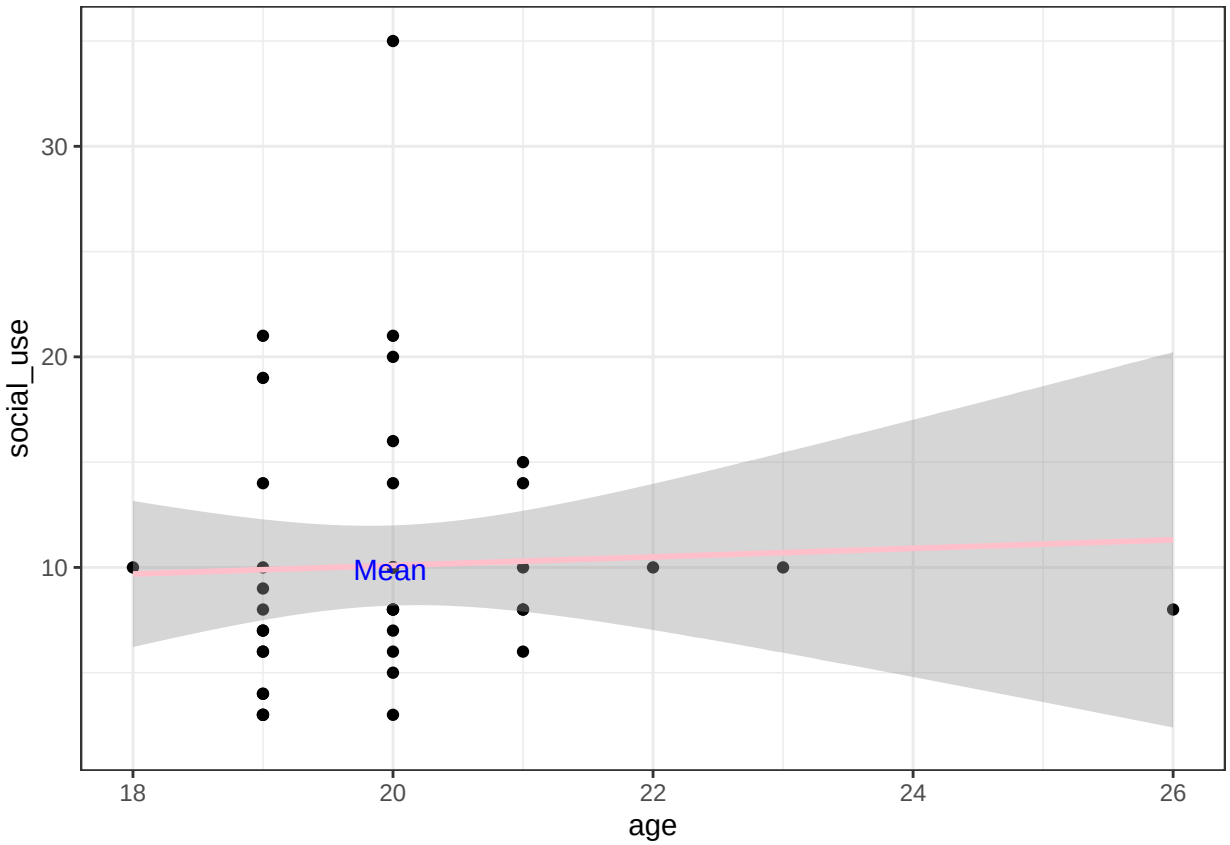
Now we can add a regression line using `geom_smooth()`.

```
class_df %>%
  ggplot(aes(x = age, y = social_use)) +
  geom_point() +
   geom_smooth(method = "lm", col = "pink") +
  annotate("text", x = mean(class_df$age, na.rm = TRUE),
           y = mean(class_df$social_use, na.rm = TRUE),
           label = "Mean", col = "blue") +
  theme_bw()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 2 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_point()`).
```



Notice how the regression line goes through the mean of X and Y.

Multiple regression

Now that we have learned the basics of linear regression what is multiple regression. Similarly to a bivariate regression model, a multivariate model is a difference in means estimator. However, in the multivariate model we estimate the change in the mean of the Y variable based on a linear function of multiple X variables. In order to fit a multivariate regression model we use the same function as a bivariate regression model.

```
multivariate_model <- lm(social_use ~ Gender + age, data = class_df)

summary(multivariate_model)
```

```
##
## Call:
## lm(formula = social_use ~ Gender + age, data = class_df)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -9.9519 -3.0329 -1.3194  0.8752 22.0481
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   7.7613    14.3333   0.541    0.591
## GenderWoman  -3.6325     2.2200  -1.636    0.110
## age           0.2595     0.7173   0.362    0.719
##
## Residual standard error: 6.1 on 39 degrees of freedom
```

```
##   (3 observations deleted due to missingness)
## Multiple R-squared:  0.0652, Adjusted R-squared:  0.01727
## F-statistic:  1.36 on 2 and 39 DF,  p-value: 0.2685
```

For this model each estimate (β coefficient) represents the average difference in Y given a one unit change in the X value holding the other variables constant. Therefore, we can interpret the coefficient on Gender as women use social media on average 3.63 hours less per week then men, while controlling for age (or holding age constant).

Coefficent Plots using ggplot2

Rafael Campos-Gottardo*

2025-03-22

In addition to being a powerful tool for statistical analysis R it provides acess to `ggplot2` a package designed for data visualization.

One of the most important skills when learning regression is how to visualize the results from a regression model. Although you can present regression results in a table, these results are harder to interpret and more cumbersome for readers. Therefore, regression tables are usually presented in the appendix and various graphs are presented in the main body. In this lab guide we are going to focus on using `ggplot` to create a coefficent plot that you can use in your final paper.

Let's get started with data visualization by loading the `tidyverse` which contains `ggplot2`.

Load packages

```
# install.packages("tidyverse") # Install the package if you have not already done so.
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Now that we have ggplot loaded let's create a simply regression model to use for our visualization. Today we will use data from the Canadian Election Study (CES). In this class we have been mostly working with country-year data, however, a lot of quantitative research is done with public opinion data like the Canadian election study. I have uploaded a cleaned version of the CES for this lab.

However, notice that the data is not a `.csv` file but is a `.dta` file. Most public opinion data is uploaded as `.dta` files (`Stata` files) because it preseves the labels. In order to open `.dta` files in R we need to use the `haven` package.

```
# install.packages("haven")
library(haven)
```

Load data

Now we can use the `read_dta()` function to open the data file.

*This lab guide builds on those created by Aaron Erlich and Colin Scott. I would also like to acknowledge Brendan Szendro, Nicholas Vaxelaire, Guila Cohen, and Natalie Delmonte for their help developing this lab guide.

```
CES <- read_dta("poli311_CES.dta")
```

Now we can use the head function to inspect the data. You'll notice that there are 5 variables including vote choice, gender, satsifaction with democracy, and province of residence.

```
head(CES)
```

```
## # A tibble: 6 x 5
##     Age Province         DemSat Gender  VoteChoice
##   <dbl> <chr>             <dbl> <chr>   <chr>
## 1    57 Quebec                3 A Man   ""
## 2    22 British Columbia      3 A Woman "NDP"
## 3    28 British Columbia      3 A Woman ""
## 4    29 Ontario               4 A Woman ""
## 5    41 Quebec                3 A Woman "NDP"
## 6    63 Quebec                3 A Woman "Bloc Quebecois"
```

Clean data

Before we fit a regression model we need to clean up the vote choice variable. Using the unique function we can see that there is a blank category. We want to remove the blank (" ") category and also make the Liberal Party the reference category.

```
unique(CES$VoteChoice)
```

```
## [1] ""                   "NDP"                "Bloc Quebecois"
## [4] "Conservative Party" "Liberal Party"      "Another Party"
## [7] "Green Party"
```

```
CES <- CES %>%
  mutate(VoteChoice = replace(VoteChoice, VoteChoice == "", NA), # Replace blank party with NA
         VoteChoice = factor(VoteChoice, levels = c(
           "Liberal Party", "Conservative Party", "NDP",
           "Bloc Quebecois", "Green Party", "Another Party")
           ) # Make the Liberal Party the reference category
         )
```

Fit a regression model

```
model_1 <- lm(DemSat ~ Age + Gender + VoteChoice, data = CES)
```

Now that we have fit a regression model regressing satisfaction with democracy on Age, Gender, and Vote Choice. After we fit the regression model we need to extract the coefficients from it to make a coefficient plot. If we look at the `class` of our model object we can see that it is an `lm` object and `ggplot` requires a data object as the first argument.

```
class(model_1)
```

```
## [1] "lm"
```

Create a data frame from our model

In order to create a data object from our model, we can use the `tidy()` function from the `broom` package.

```
# install.packages("broom")
library(broom)
```

```r
# Create a data frame (tibble) from the regression model

model_1_df <- tidy(model_1,
                   # We want to specify that we want a confidence interval
                   # for error bars
                   conf.int = TRUE)
```

Now that we have created a data set we can manipulate it to clean up the code for graphing and create a coefficient plot.

1. First, we need to use the `filter()` function to remove unwanted terms like the intercept. We would also filter out the coefficients for fixed effects (e.g. country, year, etc).
2. Second, we want to rename our terms to make the graph more presentable. Finally, we want to make the term variable a factor to make sure they display in the right order. The y axis renders from bottom to top so we have to specify the variables backwards in the `factor()` function.

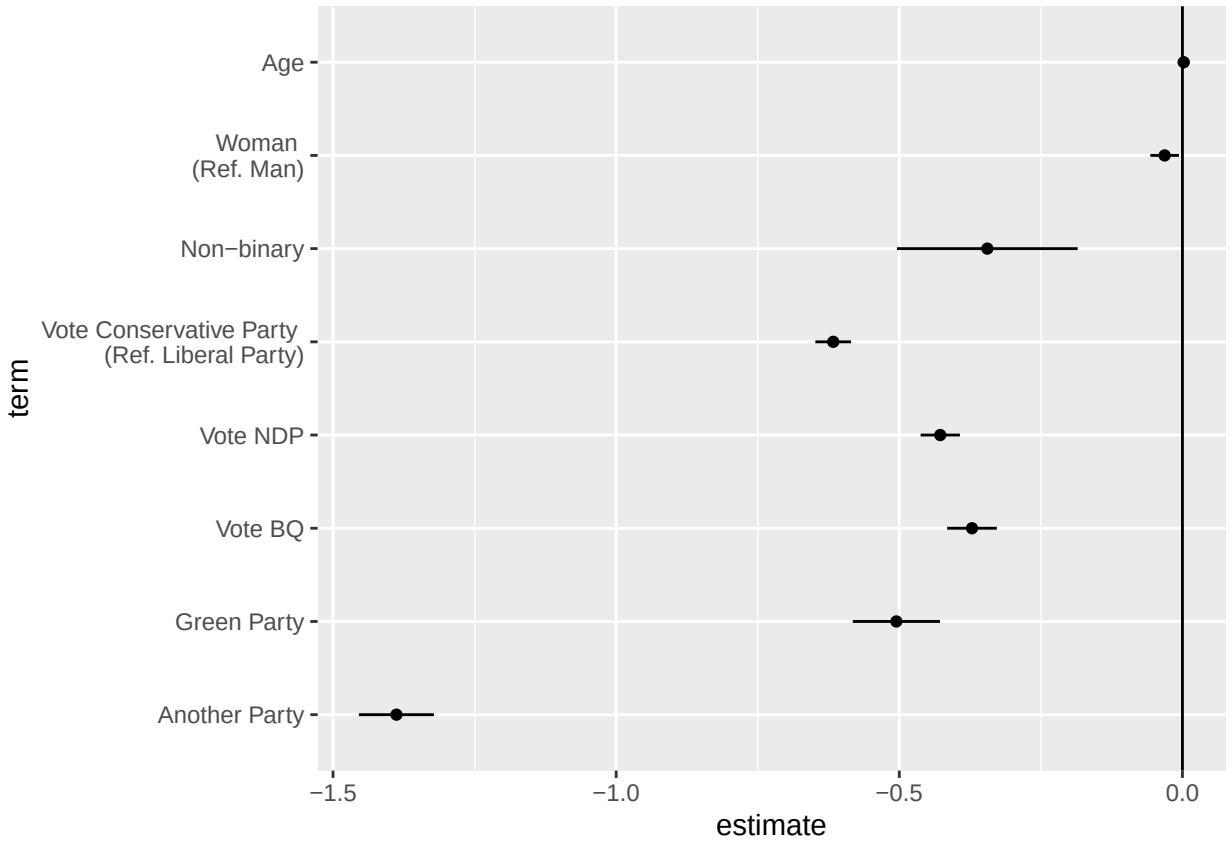
```r
model_1_df <- model_1_df %>%
  filter(term != "(Intercept)") %>% # remove the intercept
  mutate(term = case_match(term, # Rename the terms to make a cleaner plot
                           "Age" ~ "Age",
                           "GenderA Woman" ~ "Woman \n (Ref. Man)",
                           # \n puts the text after on the next line
                            "GenderNon-binary/Other Gender" ~ "Non-binary",
                            "VoteChoiceConservative Party" ~
                             "Vote Conservative Party \n (Ref. Liberal Party)",
                          "VoteChoiceNDP" ~ "Vote NDP",
                          "VoteChoiceBloc Quebecois" ~ "Vote BQ",
                          "VoteChoiceGreen Party" ~ "Green Party",
                          "VoteChoiceAnother Party" ~ "Another Party"
                           ),
         term = factor(term, levels = c("Another Party", "Green Party", "Vote BQ",
                                        "Vote NDP",
                                        "Vote Conservative Party \n (Ref. Liberal Party)",
                                        "Non-binary",
                                        "Woman \n (Ref. Man)",
                                        "Age"
                                        )))
```

Create a coefficent plot

Now that we have cleaned our dataset we can create a coefficient plot. Our x value is going to be our estimate, our x value is the variables (or term), and the xmin and xmax are our confidence intervals for our error bars.

The two geoms we use are `geom_point()` for the coefficient estimates and `geom_linerange()` for the confidence intervals. The x and y values are for `geom_point()` and the xmin and xmax are for `geom_linerange()`. However, we specify all of them in the aesthetics function (`aes()`) for `ggplot()`. Finally, we want to add a vertical line at 0 to see if the 95% confidence intervals for the regression coefficients touch 0. If the 95% confidence interval touches 0 then the coefficient on that variable is not statistically signficiant. To add this line we use `geom_vline()` and specify that the xintercept is at 0.

```r
model_1_df %>%
  ggplot(aes(x = estimate, y = term, xmin = conf.low, xmax = conf.high)) +
  geom_point() +
  geom_linerange() +
  geom_vline(xintercept = 0)
```



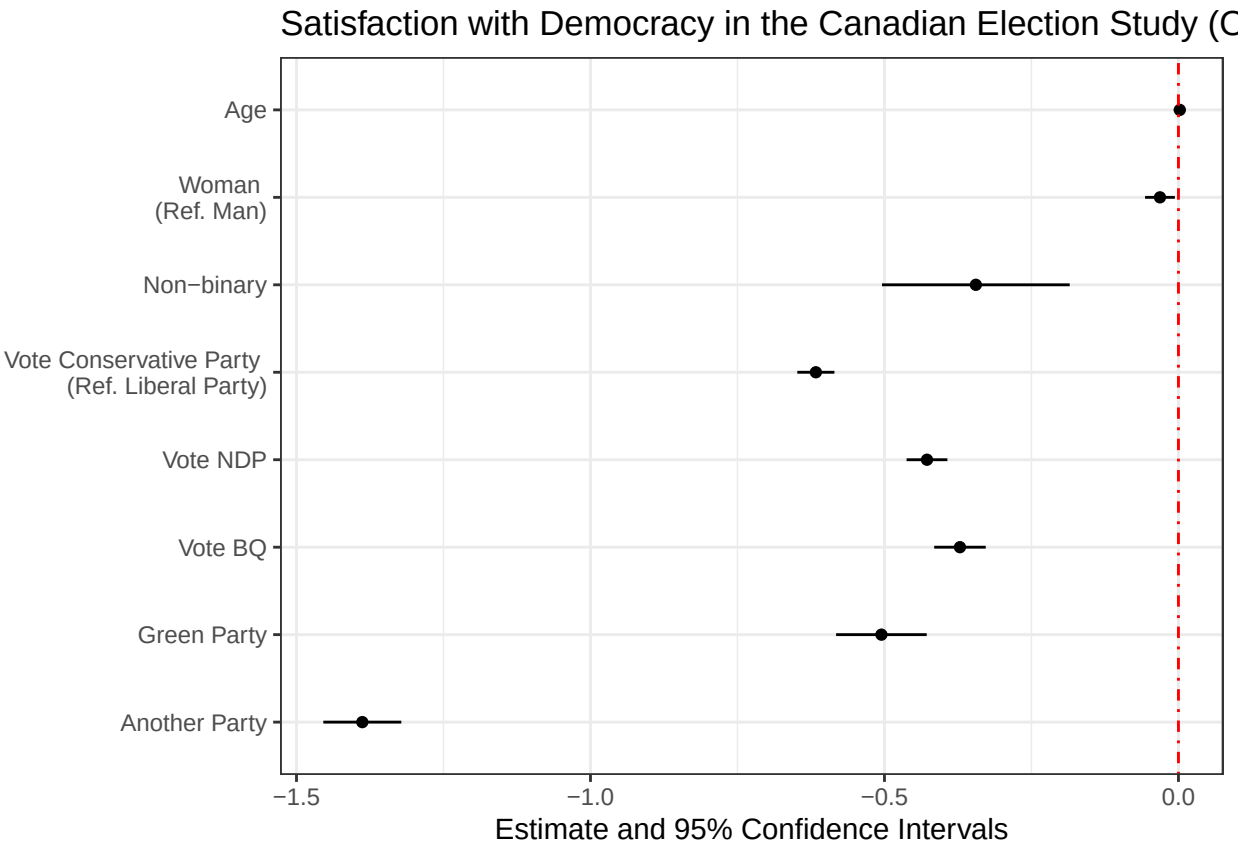
Now that we have a basic coefficient plot we can clean it up.

We want to colour our coefficients black if they are statistically significant and red if they are not, change our labels, change the line type for our x intercept, and change the theme.

Table 1: Most Common Line types ("lty")

Line type	Name
1	"solid"
2	"dashed"
3	"dotted"
4	"dotdash"

```
model_1_df %>%
  ggplot(aes(x = estimate, y = term, xmin = conf.low, xmax = conf.high)) +
  geom_point(colour = ifelse(model_1_df$p.value < 0.05, "black", "red")) +
  geom_linerange(colour = ifelse(model_1_df$p.value < 0.05, "black", "red")) +
  geom_vline(xintercept = 0,
             col = "red", # col is short for colour
             lty = 4) + # lty is short for linetype and lty 4 is a dashed and dotted line
  labs(title = "Satisfaction with Democracy in the Canadian Election Study (OLS Model)",
       x = "Estimate and 95% Confidence Intervals",
       y = NULL) + # We do not need a y axis label
  theme_bw()
```



Now we have a complete coefficient plot that we can present in a quantitative research paper.

Regression Tables

In addition to the coefficient graph, you should present your results in a regression table in the appendix of your paper. However, you should not just copy the regression output from R. To make R output a nice regression table we can use the `modelsummary()` function from the `modelsummary` package. First, we need to install and load the package.

```
# install.packages("modelsummary")
library(modelsummary)
```

```
## `modelsummary` 2.0.0 now uses `tinytable` as its default table-drawing
##   backend. Learn more at: https://vincentarelbundock.github.io/tinytable/
##
## Revert to `kableExtra` for one session:
##
##   options(modelsummary_factory_default = 'kableExtra')
##   options(modelsummary_factory_latex = 'kableExtra')
##   options(modelsummary_factory_html = 'kableExtra')
##
## Silence this message forever:
##
##   config_modelsummary(startup_message = FALSE)
```

Now we can use the `modelsummary()` function to display our regression model. We simply have to name our model in the function and then specify `stars = TRUE`. The stars argument adds the stars that indicate the significance level from the regression output produced by the `summary()` function.

	(1)
(Intercept)	3.116***
	(0.025)
Age	0.002***
	(0.000)
GenderA Woman	−0.031*
	(0.013)
GenderNon-binary/Other Gender	−0.345***
	(0.081)
VoteChoiceConservative Party	−0.617***
	(0.016)
VoteChoiceNDP	−0.428***
	(0.018)
VoteChoiceBloc Quebecois	−0.372***
	(0.022)
VoteChoiceGreen Party	−0.505***
	(0.039)
VoteChoiceAnother Party	−1.388***
	(0.034)
Num.Obs.	12 292
R2	0.180
R2 Adj.	0.179
AIC	25 824.2
BIC	25 898.4
Log.Lik.	−12 902.111
RMSE	0.69

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

```
modelsummary(model_1, stars = TRUE)
```

You will notice that we also need to clean up our variables in our model summary. We can use the map argument to change the variable names and order. We can use the same code that we used in the `case_match()` function for the map argument we just need to change the ~ to an equals sign.

```
modelsummary(model_1, stars = TRUE,
             coef_map = c("Age" = "Age",
                          "GenderA Woman" = "Woman (Ref. Man)",
                            "GenderNon-binary/Other Gender" = "Non-binary",
                            "VoteChoiceConservative Party" =
                             "Vote Conservative Party (Ref. Liberal Party)",
                          "VoteChoiceNDP" = "Vote NDP",
                          "VoteChoiceBloc Quebecois" = "Vote BQ",
                          "VoteChoiceGreen Party" = "Green Party",
                          "VoteChoiceAnother Party" = "Another Party"
```

	(1)
Age	0.002***
	(0.000)
Woman (Ref. Man)	−0.031*
	(0.013)
Non-binary	−0.345***
	(0.081)
Vote Conservative Party (Ref. Liberal Party)	−0.617***
	(0.016)
Vote NDP	−0.428***
	(0.018)
Vote BQ	−0.372***
	(0.022)
Green Party	−0.505***
	(0.039)
Another Party	−1.388***
	(0.034)
Num.Obs.	12 292
R2	0.180
R2 Adj.	0.179
AIC	25 824.2
BIC	25 898.4
Log.Lik.	−12 902.111
RMSE	0.69

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

```
))
```

Finally, if you want to display multiple models in the same table you can feed in multiple models using the `list()` function: `modelsummary(list(model1, model2), stars = TRUE)`.

Advanced Regression: Interaction Terms, Polynomial Terms, and Fixed Effects

Rafael Campos-Gottardo*

2025-03-26

Learning Objectives

- Understand interactions terms and how to interpret marginal effects.
- Visualize polynomial terms in regression.
- Understand fixed effects as isolating within variation by removing between variation.

Before Lab

Before your lab this week please ensure that you have read the lab guide and downloaded the data into your project folder you will also be more prepared for the lab if you complete the following:

- Read pages 19-22 of the Franzese and Kam reading (on *mycourses*).
- Read section 13.2.2 Polynomials of *The Effect*.
- Read Chapter 16 of the *The Effect*.

Getting started

Before we start we need to install the marginal effects package. This package is one of the most important R packages for summarizing and interpreting complex statistical models.[1]

```
#install.packages("marginaleffects")
library(marginaleffects)
```

This week we are going to use our class version of the Canadian Election Study (CES) again. Let's load the data set and then pick a model we wish to estimate (remember the dataset is in the `Stata` format so we need to use the `haven` package to load it).

```
library(haven)
CES <- read_dta("poli311_CES.dta")
head(CES)
```

```
## # A tibble: 6 x 6
##     Age Province         DemSat Gender  VoteChoice       Ideology
##   <dbl> <chr>             <dbl> <chr>   <chr>               <dbl>
## 1    57 Quebec                3 A Man   ""                      6
## 2    22 British Columbia      3 A Woman "NDP"                   3
## 3    28 British Columbia      3 A Woman ""                      5
## 4    29 Ontario               4 A Woman ""                      0
## 5    41 Quebec                3 A Woman "NDP"                   4
## 6    63 Quebec                3 A Woman "Bloc Quebecois"        0
```

*This lab guide builds on those created by Aaron Erlich and Colin Scott. I would also like to acknowledge Brendan Szendro, Nicholas Vaxelaire, Guila Cohen, and Natalie Delmonte for their help developing this lab guide.

[1] As an added bonus it was developed by a local professor at the University of Montreal - Vincent Arel-Bundock.

For ease of demonstration we are first going to filter out the non-binary respondents. We have done this a number of times in lab so try to do this code on your own before looking at the answer.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
CES <- CES %>%
 filter(Gender %in% c("A Man", "A Woman"))
```

Let us now estimate the relationship between age and ideology while controlling for Gender and satisfaction with democracy. We can formally write this model in the following way:[2]

$$Y_i = \beta_0 + \beta_1 Age_i + \beta_2 Gender_i + \beta_3 DemSat + \varepsilon_i$$

Linear regression model

Now that we have prepared our data set we can estimate our basic linear regression model.

```
model_1 <- lm(Ideology ~ Age + Gender + DemSat, data = CES)
```

We can summarize our model using the `modelsummary` package discussed in last week's lab guide.

```
#install.packages("modelsummary")
library(modelsummary)
```

```
## `modelsummary` 2.0.0 now uses `tinytable` as its default table-drawing
##   backend. Learn more at: https://vincentarelbundock.github.io/tinytable/
##
## Revert to `kableExtra` for one session:
##
##   options(modelsummary_factory_default = 'kableExtra')
##   options(modelsummary_factory_latex = 'kableExtra')
##   options(modelsummary_factory_html = 'kableExtra')
##
## Silence this message forever:
##
##   config_modelsummary(startup_message = FALSE)
```

```
modelsummary(model_1, stars = TRUE)
```

We can see that in this model age is significantly related to satisfaction with democracy where older individuals are right-wing. Specifically, a one-year increase in age is associated with a 0.014 unit increase in being more right-wing. Additionally, women are less right-wing than men and those who are more satisfied with democracy are less right-wing. However, we may have reason to believe that the relationship between age and ideology varies by gender. In other words the relationship between age and ideology is conditional on a respondent's gender.

[2] For you assignments we expect that you write out your main model in this format.

	(1)
(Intercept)	5.365***
	(0.087)
Age	0.014***
	(0.001)
GenderA Woman	−0.360***
	(0.035)
DemSat	−0.287***
	(0.023)
Num.Obs.	17 697
R2	0.028
R2 Adj.	0.028
AIC	79 874.1
BIC	79 913.0
Log.Lik.	−39 932.052
RMSE	2.31

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

Interaction Terms

One way we can test whether the relationship between age and ideology varies based on gender is by doing sub-group analysis.

We can fit a separate regression model for men and women.

```
# Men
model_2_men <- lm(Ideology ~ Age + DemSat, data = CES %>% filter(Gender == "A Man"))
# Notice how we can use the pipe within a function to make filtering easier
model_2_women <- lm(Ideology ~ Age + DemSat, data = CES %>% filter(Gender == "A Woman"))
```

Now that we have fitted these two models we can use the `modelsummary()` function to display them next to each other along with our original model.

```
modelsummary(list("Pooled Model" = model_1,
                  # You can name your models to clean up your output
                  "Men Only" = model_2_men,
                  "Women Only" = model_2_women),
             stars = TRUE)
```

In this model we can see that age has a higher correlation with ideology for women then men with the age coefficient being larger for women ($0.019 > 0.010$). However, by doing a sub-group analysis we do not know whether this difference in coefficients (effect size if this was a causal study) is statistically significant. In order to determine if this difference is statistically significant we can fit a new model using an interaction term.

This model can be formally written as follows:

$$Y_i = \beta_0 + \beta_1 Age_i + \beta_2 Gender_i + \beta_3 (Age_i \times Gender_i) + \beta_4 DemSat + \varepsilon_i$$

This model allows the slope (coefficient) of both Age and Gender to vary depending on the value of the other.

	Pooled Model	Men Only	Women Only
(Intercept)	5.365***	5.638***	4.801***
	(0.087)	(0.117)	(0.121)
Age	0.014***	0.010***	0.019***
	(0.001)	(0.001)	(0.001)
GenderA Woman	−0.360***		
	(0.035)		
DemSat	−0.287***	−0.288***	−0.287***
	(0.023)	(0.030)	(0.035)
Num.Obs.	17 697	8681	9016
R2	0.028	0.014	0.023
R2 Adj.	0.028	0.014	0.023
AIC	79 874.1	38 548.6	41 271.0
BIC	79 913.0	38 576.9	41 299.4
Log.Lik.	−39 932.052	−19 270.311	−20 631.485
RMSE	2.31	2.23	2.39

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

In order to, implement an interaction using R we add a multiplication symbol between the variables we want to interact in our model. [Note that you could also use the colon symbol (:) to implement an interaction, however, R will only provide a coefficient for the interaction and not for the individual variables.]

```
interaction_model <- lm(Ideology ~ Age*Gender + DemSat, data = CES)
```

Now we can add this model to our model summary.

```
modelsummary(list("Pooled Model" = model_1,
                  "Men Only" = model_2_men,
                  "Women Only" = model_2_women,
                  "Interaction Model" = interaction_model),
             stars = TRUE)
```

What do you notice about the output from this model?

1. Notice that we get a new coefficient in this model `Age x GenderA Woman` this coefficient is called an interaction term and tells us whether the difference in our coefficients is statistically significant. Since our interaction term has three stars we can say that there is statistically significant difference in the effect of age on ideology between men and women.
2. Notice that our age coefficient is the same as the age coefficient for our men only model. This is because the age coefficient represents the relationship between age and ideology for the baseline group for our interaction term (in this case men). In order to get the coefficient for women we need to add the interaction term to the age coefficient $0.010 + 0.009 = 0.019$. Adding these coefficients provides the coefficient for the relationship between age and ideology for women.

Another way we can present these results is by using `slopes()` function from the marginal effects package. This function provides the coefficients for age for each level of gender.

```
slopes(interaction_model, variables = "Age", by = "Gender")
```

```
##
##  Term    Contrast  Gender Estimate Std. Error     z Pr(>|z|)     S   2.5 %
```

	Pooled Model	Men Only	Women Only	Interaction Model
(Intercept)	5.365***	5.638***	4.801***	5.637***
	(0.087)	(0.117)	(0.121)	(0.107)
Age	0.014***	0.010***	0.019***	0.010***
	(0.001)	(0.001)	(0.001)	(0.002)
GenderA Woman	−0.360***			−0.834***
	(0.035)			(0.114)
DemSat	−0.287***	−0.288***	−0.287***	−0.287***
	(0.023)	(0.030)	(0.035)	(0.023)
Age × GenderA Woman				0.009***
				(0.002)
Num.Obs.	17 697	8681	9016	17 697
R2	0.028	0.014	0.023	0.029
R2 Adj.	0.028	0.014	0.023	0.029
AIC	79 874.1	38 548.6	41 271.0	79 857.0
BIC	79 913.0	38 576.9	41 299.4	79 903.7
Log.Lik.	−39 932.052	−19 270.311	−20 631.485	−39 922.510
RMSE	2.31	2.23	2.39	2.31

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

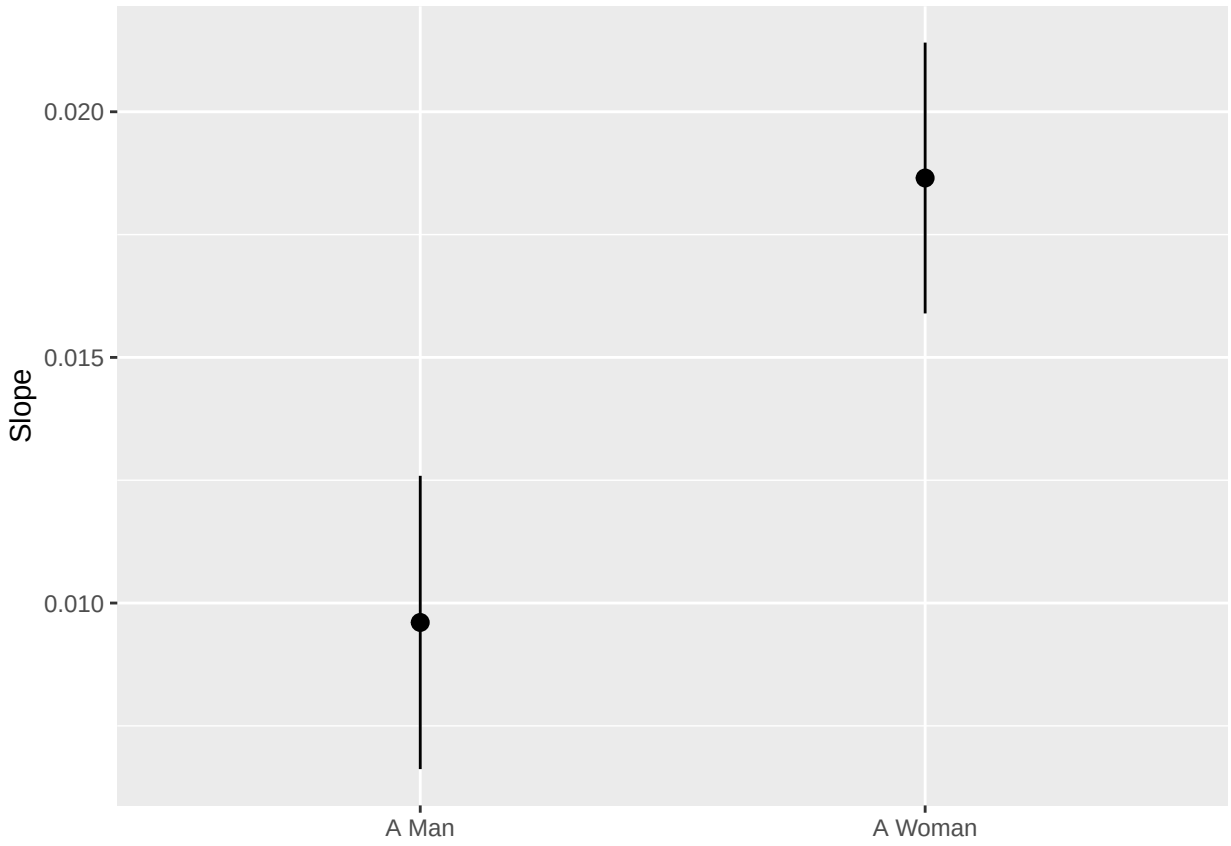
```
##   Age mean(dY/dX) A Man    0.00961    0.00152  6.31   <0.001  31.7 0.00662
##   Age mean(dY/dX) A Woman  0.01865    0.00141 13.26   <0.001 130.9 0.01590
##  97.5 %
##  0.0126
##  0.0214
##
## Columns: term, contrast, Gender, estimate, std.error, statistic, p.value, s.value, conf.low, conf.hi
## Type:  response
```

Notice how we get the same coefficients as in our sub-group models and our interaction model.

Thought Question: What happens if we put "Gender" in the variables argument and "Age" in the by argument?

We can also use the `plot_slopes()` function from the marginal effects package to plot these regression coefficients and the difference. FYI this function produces a "`gg`" object that we can add additional `ggplot` arguments to.

```
plot_slopes(interaction_model, variables = "Age", by = "Gender")
```



Polynomial terms

Another transformation we can include in our regression models is polynomial terms. In a standard linear regression we assume that the relationship between our X and Y variables is linear. However, for many social phenomena the relationship between X and Y is not linear. Therefore, we can include a polynomial term to let our regression line curve. One variable that is often not linearly related to dependent variables is age.

We can fit a new model that examines the relationship between age and ideology with a polynomial term that can be formally expressed as so:

$$Y_i = \beta_0 + \beta_1 Age_i + \beta_2 Age_i^2 + \beta_3 Gender_i + \beta_4 DemSat + \varepsilon_i$$

In order to fit this model we will use the `I()` function within our regression model which allows us to transform variables within the model using mathematical operators. It is important to also keep the original age coefficient in our model to retain the original linear relationship.

```
polynomial_model <- lm(Ideology ~ Age + I(Age^2) + Gender + DemSat, data = CES)
```

Let's also add this model to our model summary.

```
modelsummary(list("Pooled Model" = model_1,
                  "Men Only" = model_2_men,
                  "Women Only" = model_2_women,
                  "Interaction Model" = interaction_model,
                  "Curvilinear Model" = polynomial_model),
             stars = TRUE)
```

This model indicates that as age increases individuals get become more right-wing, and that this relationship becomes stronger as people get older. One way to better understand this model is to graph the predictions

	Pooled Model	Men Only	Women Only	Interaction Model	Curvilinear Model
(Intercept)	5.365***	5.638***	4.801***	5.637***	4.650***
	(0.087)	(0.117)	(0.121)	(0.107)	(0.167)
Age	0.014***	0.010***	0.019***	0.010***	0.046***
	(0.001)	(0.001)	(0.001)	(0.002)	(0.006)
GenderA Woman	−0.360***			−0.834***	−0.356***
	(0.035)			(0.114)	(0.035)
DemSat	−0.287***	−0.288***	−0.287***	−0.287***	−0.282***
	(0.023)	(0.030)	(0.035)	(0.023)	(0.023)
Age × GenderA Woman				0.009***	
				(0.002)	
I(Age^2)					0.000***
					(0.000)
Num.Obs.	17 697	8681	9016	17 697	17 697
R2	0.028	0.014	0.023	0.029	0.029
R2 Adj.	0.028	0.014	0.023	0.029	0.029
AIC	79 874.1	38 548.6	41 271.0	79 857.0	79 850.9
BIC	79 913.0	38 576.9	41 299.4	79 903.7	79 897.6
Log.Lik.	−39 932.052	−19 270.311	−20 631.485	−39 922.510	−39 919.439
RMSE	2.31	2.23	2.39	2.31	2.31

+ $p < 0.1$, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

from it. First we need to create predictions using the predict function.

For ease we are going to re-run the model without controls. However, if you want to include controls when predicting the common practice is to hold the control variables at their mean or median.

```
polynomial_model_2 <- lm(Ideology ~ Age + I(Age^2), data = CES)
summary(polynomial_model_2)
```

```
##
## Call:
## lm(formula = Ideology ~ Age + I(Age^2), data = CES)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -5.3757 -1.5050 -0.1649  1.6618  5.6422
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.5011255  0.1487996  23.529  < 2e-16 ***
## Age          0.0519154  0.0062969   8.245  < 2e-16 ***
## I(Age^2)    -0.0003594  0.0000617  -5.825 5.81e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.325 on 17962 degrees of freedom
##   (2879 observations deleted due to missingness)
## Multiple R-squared:  0.01513,    Adjusted R-squared:  0.01502
## F-statistic: 137.9 on 2 and 17962 DF,  p-value: < 2.2e-16
```

Notice that when not controlling for Gender and satisfaction with democracy the Age squared coefficient becomes negative. Therefore, our graph is going to have an inverted U-shape.

To create predictions we use the `predict()` function.

```
polynomial_predictions <- predict(polynomial_model_2,
                                  newdata = data.frame(Age = 18:99))

polynomial_predictions
```

```
##        1        2        3        4        5        6        7        8
## 4.319146 4.357762 4.395660 4.432839 4.469298 4.505039 4.540061 4.574365
##        9       10       11       12       13       14       15       16
## 4.607949 4.640814 4.672961 4.704389 4.735098 4.765088 4.794359 4.822911
##       17       18       19       20       21       22       23       24
## 4.850745 4.877859 4.904255 4.929932 4.954890 4.979129 5.002649 5.025450
##       25       26       27       28       29       30       31       32
## 5.047533 5.068896 5.089541 5.109467 5.128674 5.147162 5.164932 5.181982
##       33       34       35       36       37       38       39       40
## 5.198314 5.213927 5.228820 5.242995 5.256452 5.269189 5.281207 5.292507
##       41       42       43       44       45       46       47       48
## 5.303088 5.312949 5.322092 5.330516 5.338222 5.345208 5.351476 5.357024
##       49       50       51       52       53       54       55       56
## 5.361854 5.365965 5.369357 5.372030 5.373984 5.375220 5.375736 5.375534
##       57       58       59       60       61       62       63       64
## 5.374613 5.372973 5.370614 5.367537 5.363740 5.359224 5.353990 5.348037
##       65       66       67       68       69       70       71       72
## 5.341365 5.333974 5.325864 5.317036 5.307488 5.297222 5.286237 5.274532
```

```
##       73       74       75       76       77       78       79       80
## 5.262109 5.248968 5.235107 5.220527 5.205229 5.189212 5.172476 5.155021
##       81       82
## 5.136847 5.117954
```

Now we have predicted values of ideology for individuals aged 18 to 99. Lets create a dataframe so that we can graph this using `ggplot`.

```
predictions_df <- data.frame(Age = 18:99,
                             Predicted_Ideology = polynomial_predictions)

head(predictions_df)
```

```
##   Age Predicted_Ideology
## 1  18           4.319146
## 2  19           4.357762
## 3  20           4.395660
## 4  21           4.432839
## 5  22           4.469298
## 6  23           4.505039
```

Now we can graph this curvilinear relationship.

```
predictions_df %>%
  ggplot(aes(x = Age, y = Predicted_Ideology)) +
  geom_line(colour = "forestgreen") +
  labs(title = "Curvilinear Relationship Between Age and Ideology",
       x = "Age",
       y = "Predicted Ideology") +
  theme_bw()
```

Here you can see that as individuals get older they become more right-wing until they reach their 70s where they start becoming more left-wing again.

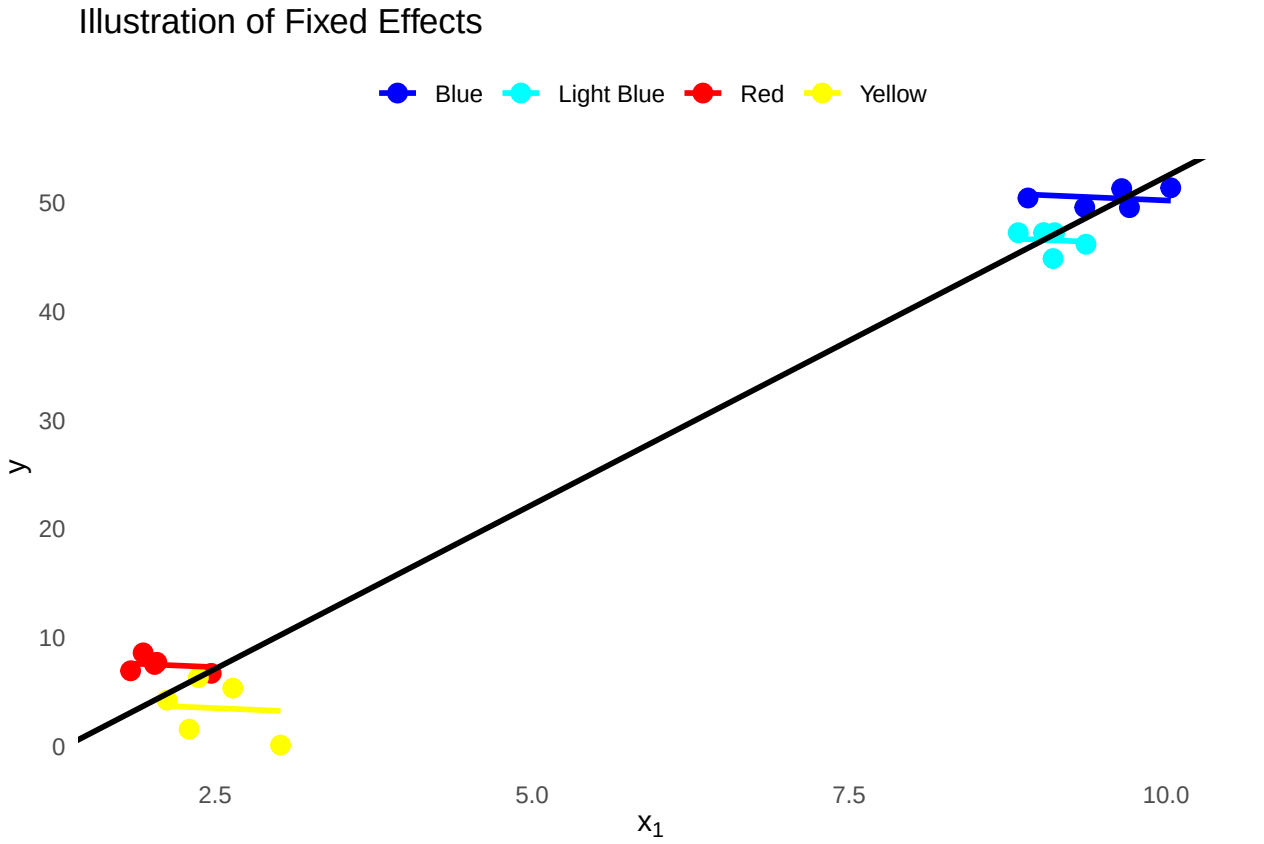
Fixed effects

Before we talk about fixed effects we need to understand between and within variation.

1. Between(-group) variation: The difference across units (Comparing two countries, provinces, individuals, etc.).
2. Within(-group) variation: The differences within a unit across time (changes in levels of polarization within Alberta from 2001-2025). We can also measure within variation without panel data by ignoring the fixed characteristics of a given unit without the time component.

Fixed effects help isolate within-unit variation by removing between-unit variation. We do this by adding unit-level dummy variables (e.g., provinces):

Fixed effects are most common when using data that includes units and time. For example, using fixed effects is useful for country-year data. The inclusion of fixed effects allows a researcher to isolate within variation. For example we might have a panel of Canadian voters and want isolate the relationship between economic globalization and levels of polarization. However, different provinces have different base levels of polarization that do not vary over time. We do not want these time invariant factors to interfere with our analysis so we include fixed effects to remove the between unit variation. Therefore, we can understand fixed effects as helping us remove between variation so that we can isolate the within variation.



This graph demonstrates an extreme example of how our estimate of the relationship between an X and Y variable would be biased due to between variation if we did not include fixed effects in our model. Notice that the "main effect" is positive but the slope for the within variation is negative.

In order to add fixed effects to a regression model we can add dummy variables for each unit. For our model estimating the relationship between age and ideology we can add province fixed effects by including our province variable as a factor variable. This model can be formally written as follows where α_p represents province-level fixed effects (notice they are for p not respondent i):

$$Y_i = \beta_1 Age_i + \beta_2 Age_i^2 + \beta_3 Gender_i + \beta_4 DemSat + \alpha_p + \varepsilon_i$$

We can use R to add fixed effects as follows.

```
fixed_effects_model <- lm(Ideology ~ Age + Gender + DemSat + Province, data = CES)
```

Now we can add our final model to our model summary.

```
modelsummary(list("Pooled Model" = model_1,
                  "Interaction Model" = interaction_model,
                  "Curvilinear Model" = polynomial_model,
                  "Fixed Effects Model" = fixed_effects_model),
             stars = TRUE)
```

The final model only shows us the relationship between age and ideology within each province. Notice that the coefficient is the same as in our baseline model. This indicates that the relationship between age and ideology does not vary across province. However, the coefficients for gender and satisfaction with democracy are different indicating that this relationship does vary across province.

	Pooled Model	Interaction Model	Curvilinear Model	Fixed Effects Model
(Intercept)	5.365***	5.637***	4.650***	5.628***
	(0.087)	(0.107)	(0.167)	(0.097)
Age	0.014***	0.010***	0.046***	0.014***
	(0.001)	(0.002)	(0.006)	(0.001)
GenderA Woman	−0.360***	−0.834***	−0.356***	−0.383***
	(0.035)	(0.114)	(0.035)	(0.035)
DemSat	−0.287***	−0.287***	−0.282***	−0.261***
	(0.023)	(0.023)	(0.023)	(0.023)
Age × GenderA Woman		0.009***		
		(0.002)		
I(Age^2)			0.000***	
			(0.000)	
ProvinceBritish Columbia				−0.362***
				(0.071)
ProvinceManitoba				−0.127
				(0.103)
ProvinceNew Brunswick				−0.581***
				(0.138)
ProvinceNewfoundland and Labrador				−0.214
				(0.194)
ProvinceNorthwest Territories				−1.120+
				(0.667)
ProvinceNova Scotia				−0.636***
				(0.122)
ProvinceNunavut				0.726
				(1.153)
ProvinceOntario				−0.275***
				(0.057)
ProvincePrince Edward Island				0.483
				(0.323)
ProvinceQuebec				−0.480***
				(0.059)
ProvinceSaskatchewan				0.216+
				(0.130)
ProvinceYukon				−0.299
				(0.463)
Num.Obs.	17 697	17 697	17 697	17 697
R2	0.028	0.029	0.029	0.034
R2 Adj.	0.028	0.029	0.029	0.033
AIC	79 874.1	79 857.0	79 850.9	79 784.0
BIC	79 913.0	79 903.7	79 897.6	79 916.3
Log.Lik.	−39 932.052	−39 922.510	−39 919.439	−39 874.996
RMSE	2.31	2.31	2.31	2.30

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

Key Takeaways

- Use *interaction terms* to capture how the effect of one variable depends on another.
- Use *polynomial terms* to model non-linear relationships. These are mostly used for curviliear relationships.
- Use *fixed effects* to control unobserved differences across units and focus on within-group variation.